



**UNIVERSIDADE ESTADUAL DA PARAÍBA
CENTRO DE CIÊNCIA E TECNOLOGIA - CCT
DEPARTAMENTO DE COMPUTAÇÃO
CURSO DE BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO
PROFESSOR DR.: FÁBIO LUIZ LEITE JÚNIOR**

**Adrielly Carla Ferreira de Melo
Mirelle Casimiro Silvino
Pedro Henrique da Silva Sales
Rui Fernando do Nascimento Filho**

PROJETO DE ANÁLISE DE ALGORITMOS: SUDOKU SOLVER.

CAMPINA GRANDE - PB

2026

RESUMO

Este trabalho apresenta o desenvolvimento e a análise comparativa de diferentes algoritmos para a resolução do problema do Sudoku, utilizando como base os algoritmos de Backtracking, Branch and Bound e Programação Dinâmica. O projeto foi desenvolvido com o objetivo de investigar o comportamento dessas estratégias de resolução em tabuleiros de diferentes dimensões e níveis de dificuldade, avaliando aspectos como tempo de execução, quantidade de nós visitados, número de retrocessos, podas realizadas, profundidade máxima e consumo de memória. Foram utilizados tabuleiros de diferentes tamanhos, permitindo observar como o aumento da dimensão e da dificuldade influencia o desempenho dos algoritmos. A partir dos experimentos realizados, foi possível identificar diferenças significativas entre as estratégias, principalmente em relação à quantidade de estados explorados e ao tempo necessário para encontrar uma solução. De maneira geral, o Backtracking apresentou melhor desempenho, especialmente nos tabuleiros de menor dificuldade, enquanto os tabuleiros maiores e mais complexos exigiam maior esforço computacional. A dificuldade mais significativa foi observada nos tabuleiros de maior dimensão e maior nível de dificuldade, nos quais o espaço de busca aumenta consideravelmente e as estratégias precisam explorar um número maior de possibilidades. O projeto, portanto, demonstra na prática a aplicação e comparação de técnicas de busca e otimização estudadas na disciplina de Técnicas Avançadas de Algoritmos, permitindo compreender como diferentes abordagens algorítmicas se comportam diante de um mesmo problema e como a complexidade do problema influencia diretamente o desempenho computacional.

SUMÁRIO

1. INTRODUÇÃO.....	5
2. CONTEXTUALIZAÇÃO DO PROBLEMA.....	6
2.1 Motivação.....	7
3. FUNDAMENTAÇÃO TEÓRICA.....	8
3.1. Descrição do Problema.....	8
3.2. Backtracking.....	9
3.2.1. Análise de Corretude.....	10
3.2.2. Análise de Desempenho.....	12
3.2.2.1. Espacial.....	12
3.2.2.2. Temporal.....	13
3.3. Branch and Bound.....	15
3.3.1. Análise de Corretude.....	16
3.3.2. Análise de Desempenho.....	17
3.3.2.1. Espacial.....	17
3.3.2.2. Temporal.....	18
3.4. Programação Dinâmica.....	19
3.4.1. Análise de Corretude.....	21
3.4.2. Análise de Desempenho.....	22
3.4.2.1. Espacial.....	22
3.4.2.2. Temporal.....	22
4. METODOLOGIA.....	25
4.1 Ambiente Computacional.....	25
4.1.1 Configuração dos Experimentos.....	29
4.2 Linguagem Utilizada.....	29
4.3 Estrutura de dados.....	30
4.3.1. Estrutura utilizada pelo Backtracking.....	31
4.3.2. Estruturas utilizadas pelo Branch and Bound.....	31
4.3.3. Estruturas utilizadas pela Programação Dinâmica.....	32
4.3.4. Estruturas auxiliares.....	32
4.4 Critérios Experimentais.....	33
4.4.1. Algoritmos avaliados.....	33
4.4.2. Repetição dos experimentos.....	34
4.4.3. Métricas analisadas.....	34
4.4.4. Controle das condições de execução.....	35
4.4.5. Critério de escalabilidade.....	36
4.4.6. Registro dos resultados.....	37
5. EXPERIMENTOS E RESULTADOS.....	38
5.1. Instâncias de Testes.....	38

5.2 Resultados por tamanho do tabuleiro.....	39
5.3 Resultados da escalabilidade entre tamanhos de tabuleiro.....	47
5.4 Resultados de comparação do consumo de memória.....	48
6. ANÁLISE E DISCUSSÃO.....	51
6.1 Comparação do Tempo de Execução.....	51
6.2 Comparação do Consumo de Memória.....	52
6.3 Corretude e Convergência das Soluções.....	53
6.4 Limites de Viabilidade dos Algoritmos.....	54
6.5 Impacto do Crescimento do Tamanho do Tabuleiro.....	54
6.6 Relação entre Complexidade Teórica e Desempenho Prático.....	55
6.7 Eficiência dos Mecanismos de Poda e Memoização.....	56
6.8 Custo Computacional e Benefícios das Estratégias Algorítmicas.....	57
6.9 Limitações da Coleta e Interpretação dos Dados.....	58
7. CONCLUSÃO.....	59
Desafio Extra (Bônus).....	62
REFERÊNCIAS.....	63

1. INTRODUÇÃO

No curso de Técnicas de Análise de Algoritmos, foram sugeridas atividades em grupos com a finalidade de criar soluções computacionais usando as técnicas de algoritmos ensinadas nas aulas. Cada equipe deveria escolher um dos temas indicados pelo professor e desenvolver uma solução com base nos métodos de design de algoritmos discutidos durante o curso.

A opção escolhida pela nossa equipe foi o Sudoku Solver, que se refere ao tema 15 da lista de projetos. A tarefa envolve criar um sistema que consiga resolver um tabuleiro de Sudoku automaticamente, utilizando as estratégias algorítmicas apropriadas para identificar uma solução válida.

Para solucionar o problema, diversas técnicas de design de algoritmos foram consideradas, incluindo Backtracking, Branch and Bound e Programação Dinâmica, permitindo a avaliação de suas utilizações e eficiência na tarefa de resolver o Sudoku.

Cabe destacar que, no desenvolvimento deste trabalho, foi utilizada a ferramenta de Inteligência Artificial Generativa Claude (Anthropic) como recurso de apoio, em conformidade com a RESOLUÇÃO/UEPB/CONSEPE/031/2026, que estabelece a política de uso responsável da IAGen na UEPB. A ferramenta foi empregada como auxílio na implementação da interface gráfica (frontend) do sistema, desenvolvida em Java Swing, bem como no esclarecimento de dúvidas pontuais durante a etapa de análise dos resultados obtidos. A construção metodológica, a interpretação dos dados e a redação final deste trabalho permaneceram sob total responsabilidade e controle crítico dos autores.

2. CONTEXTUALIZAÇÃO DO PROBLEMA

O Sudoku é um jogo de lógica antigo, amplamente conhecido, que consiste em preencher um tabuleiro dividido em sub-grades quadradas com números, respeitando restrições específicas: cada número deve aparecer apenas uma vez em cada linha, em cada coluna e em cada sub-grade. Na versão clássica, o tabuleiro é 9x9, dividido em nove sub-grades 3x3, preenchido com números de 1 a 9.

No contexto computacional, o Sudoku pode ser tratado como um problema de busca com restrições (CSP), no qual um algoritmo deve encontrar valores válidos para as células vazias de um tabuleiro parcialmente preenchido. Nesse problema, algumas posições já possuem números definidos, enquanto outras estão vazias e precisam ser completadas de forma que todas as regras do jogo sejam satisfeitas.

Neste projeto, o problema é abordado de forma generalizada, contemplando tabuleiros de quatro tamanhos distintos 4x4, 9x9, 16x16 e 25x25, cada um dividido em sub-grades proporcionais (2x2, 3x3, 4x4 e 5x5, respectivamente). Para cada tamanho de tabuleiro, são consideradas três instâncias de dificuldade: fácil, médio e difícil, variando de acordo com a quantidade de células preenchidas inicialmente. O tabuleiro de entrada é representado por uma matriz de números inteiros, onde o valor 0 indica uma célula vazia a ser preenchida, e o algoritmo deve produzir como saída o tabuleiro completo, sem violar nenhuma das restrições do jogo.

Do ponto de vista da análise de algoritmos, o problema de resolver Sudoku é especialmente interessante porque o espaço de busca cresce de forma acentuada à medida que o tamanho do tabuleiro aumenta, tornando a escolha da técnica algorítmica um fator determinante para a viabilidade da solução em instâncias maiores. Dessa forma, métodos de projeto de algoritmos como Backtracking, Branch and Bound e Dynamic Programming podem ser investigados e comparados não apenas quanto à correção, mas principalmente quanto à sua capacidade de escalar frente ao aumento da complexidade do problema.

2.1 Motivação

O problema de decisão associado ao Sudoku generalizado é conhecido como NP-completo como satisfação de restrições, o que o torna um objeto de estudo particularmente interessante para a disciplina de Técnicas de Análise de Algoritmos. Sua estrutura permite não apenas testar a corretude de diferentes abordagens algorítmicas, mas também observar de forma prática como cada técnica se comporta diante do crescimento do espaço de busca.

A escolha do tema é motivada principalmente pelo interesse em investigar a escalabilidade das técnicas de projeto de algoritmos: Backtracking, Branch and Bound e Programação Dinâmica ao serem aplicadas a tabuleiros de tamanhos crescentes (4x4, 9x9, 16x16 e 25x25), cada um avaliado em três níveis de dificuldade (fácil, médio e difícil). Essa variação permite observar, de forma controlada, como o desempenho de cada algoritmo se degrada (ou se mantém) à medida que o número de células, restrições e combinações possíveis aumenta exponencialmente.

Enquanto técnicas como Backtracking e Branch and Bound são naturalmente adequadas à exploração do espaço de soluções com poda, tornando-as candidatas mais robustas para tabuleiros maiores, abordagens como Programação Dinâmica não é tradicionalmente associada à resolução de CSPs, o que torna especialmente relevante observar seus limites de escalabilidade, ou a sua inviabilidade, conforme o tabuleiro cresce.

Além do valor acadêmico, o projeto permite explorar métricas concretas de desempenho, como tempo de execução, número de nós visitados e tamanho da árvore de busca, em função do tamanho do tabuleiro e do nível de dificuldade, possibilitando uma análise quantitativa e comparativa da escalabilidade de cada técnica, reforçando de forma prática os conceitos teóricos de complexidade estudados ao longo do curso.

3. FUNDAMENTAÇÃO TEÓRICA

Esta seção apresenta uma análise detalhada e descrição formal dos algoritmos de resolução implementados no sistema, abordando seus fundamentos teóricos, decisões de implementação, corretude formal e complexidade computacional. Para cada algoritmo, são examinadas separadamente a complexidade espacial e a complexidade temporal.

3.1. Descrição do Problema

Do ponto de vista formal, o Sudoku pode ser modelado como um Problema de Satisfação de Restrições (CSP — Constraint Satisfaction Problem), definido por uma tripla (X, D, C) , em que:

- **X** é o conjunto de variáveis, correspondente a cada célula do tabuleiro de dimensão $N \times N$;
- **D** é o domínio de valores possíveis para cada variável, correspondente aos inteiros de 1 a N , onde N assume os valores 4, 9, 16 ou 25 conforme a instância considerada;
- **C** é o conjunto de restrições de unicidade: nenhum valor pode se repetir na mesma linha, na mesma coluna ou na mesma subgrade $\sqrt{N} \times \sqrt{N}$.

Uma célula é dita fixa quando seu valor é fornecido como pista inicial do problema, e livre quando seu valor é 0 e precisa ser determinado pelo algoritmo de resolução. Uma atribuição é considerada consistente quando não viola nenhuma das restrições de C , e uma solução é completa quando todas as variáveis de X possuem um valor atribuído dentro do domínio D , respeitando integralmente C .

A versão generalizada do problema de decisão do Sudoku, considerando tabuleiros de dimensão variável, é conhecida como NP-completa, o que justifica a

ausência de algoritmos polinomiais exatos conhecidos para o caso geral e motiva a investigação de diferentes técnicas de projeto de algoritmos, cada uma explorando o espaço de busca de forma distinta, quanto à sua viabilidade prática. Essa característica é particularmente relevante neste trabalho, uma vez que o crescimento de N amplia o espaço de busca de forma acentuada, tornando a escolha da técnica algorítmica um fator determinante para a viabilidade da solução nas instâncias maiores.

Neste trabalho, o problema é tratado de forma generalizada, contemplando quatro dimensões distintas sendo elas: 4×4 , 9×9 , 16×16 e 25×25 , cada uma dividida em subgrades proporcionais (2×2 , 3×3 , 4×4 e 5×5 , respectivamente), com o objetivo de investigar a escalabilidade das técnicas algorítmicas frente ao aumento do tamanho do tabuleiro. Para cada dimensão, são consideradas três instâncias de dificuldade (fácil, médio e difícil). A entrada é representada por uma matriz $N \times N$ de números inteiros onde o valor 0 indica uma célula livre, e o algoritmo deve produzir como saída uma atribuição completa e consistente para todas as células, ou indicar a inexistência de solução, quando aplicável.

3.2. Backtracking

O Backtracking é uma técnica algorítmica de busca exaustiva com poda, amplamente utilizada na resolução de problemas de satisfação de restrições. A ideia central consiste em construir a solução incrementalmente, atribuindo valores a variáveis uma a uma e abandonando imediatamente qualquer atribuição parcial que viole alguma restrição, sem precisar explorar todas as extensões possíveis daquela atribuição. No contexto do Sudoku, cada célula vazia corresponde a uma variável, e as restrições são as regras de unicidade por linha, coluna e subgrade $\sqrt{N} \times \sqrt{N}$.

A implementação do sistema segue esse paradigma de forma direta. A classe `BacktrackingSolver` percorre as células vazias em ordem linear, linha por linha da esquerda para a direita, tentando inserir os valores de 1 a N em cada posição, onde N corresponde à dimensão do tabuleiro sob análise (4, 9, 16 ou 25).

Quando nenhum valor é válido para a célula atual, a última atribuição é desfeita e o algoritmo retrocede para a célula anterior. Esse processo se repete recursivamente até que todas as células estejam preenchidas, indicando que a solução foi encontrada, ou até que o espaço de busca se esgote completamente, caracterizando um jogo sem solução.

Um aspecto relevante da implementação é o suporte ao modo de visualização passo a passo, ativado pelo parâmetro booleano `visualizar` no construtor. Quando ativo, o algoritmo exibe o estado do tabuleiro a cada atribuição e a cada retrocesso, com um atraso de 150 ms entre os passos por meio de `Thread.sleep`. Esse recurso é desativado nos testes automatizados e nos experimentos em lote, onde a instância é criada com `new BacktrackingSolver(false)`, eliminando o overhead de I/O e de temporização do ciclo de medição. O nome do nível de dificuldade é recuperado dinamicamente por meio de `System.getProperty("difficulty")`, permitindo que tanto o `ExperimentRunner` quanto os testes JUnit configurem o rótulo exibido sem a necessidade de repassar o parâmetro pela cadeia de chamadas recursivas.

3.2.1. Análise de Corretude

A corretude de um algoritmo de busca com retrocesso pode ser demonstrada por três propriedades complementares: completude, validade e terminação. O algoritmo é completo para instâncias de Sudoku bem formadas, pois o espaço de busca do Sudoku $N \times N$ é finito e limitado, com no máximo $N^{(N^2)}$ estados possíveis, embora a grande maioria seja eliminada pelas restrições logo nas primeiras atribuições. Vale notar que esse limite cresce de forma extremamente acentuada com N : para $N=9$, o limite é da ordem de $9^{81} \approx 10^{77}$; para $N=25$, chega a 25^{625} , um número astronomicamente maior, o que evidencia por que a poda imposta pelas restrições é indispensável mesmo nas instâncias menores, e ainda mais crítica nas maiores.

A cada nível de recursão, o algoritmo experimenta todos os valores candidatos para a célula atual, de 1 a N , antes de retornar `false`. Se alguma

combinação de valores leva a uma solução, ela será necessariamente alcançada, uma vez que o algoritmo explora sistematicamente todas as ramificações válidas da árvore de busca sem saltar nenhum nó. A condição de parada, que consiste em retornar true quando não existem mais células com valor 0, garante que a solução só é declarada quando o tabuleiro está integralmente preenchido.

A correção do laço principal pode ser estabelecida por meio de uma invariante de laço: ao início de cada iteração sobre os valores candidatos de uma célula, o tabuleiro encontra-se em um estado parcialmente preenchido que é consistente com todas as restrições do Sudoku, ou seja, nenhuma das atribuições realizadas até aquele ponto viola unicidade de linha, coluna ou subgrade.

Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada com pistas que não se contradizem entre si. É mantida a cada iteração, pois a atribuição de um novo valor à célula corrente só ocorre após a confirmação de `SudokuValidator.isValid`, que garante que o valor não cria nenhum conflito com o estado atual; caso nenhum valor satisfaça a verificação, a célula permanece inalterada e o algoritmo retrocede, restaurando o estado consistente anterior. É satisfeita na terminação, pois quando o laço encerra com retorno true, todas as células foram preenchidas e a invariante de consistência global do tabuleiro está integralmente preservada.

Além disso, toda atribuição realizada pelo algoritmo é precedida pela verificação `SudokuValidator.isValid(tabuleiro, linha, coluna, número)`, que confirma que o valor candidato não viola nenhuma das três restrições simultaneamente: unicidade na linha, unicidade na coluna e unicidade na subgrade $\sqrt{N} \times \sqrt{N}$.

Como essa verificação ocorre antes de cada atribuição, qualquer estado parcial mantido durante a execução permanece consistente com as regras do jogo. Consequentemente, quando o algoritmo retorna true, o tabuleiro está completo e cada uma das N^2 células satisfaz todas as restrições. Essa propriedade é formalmente validada pelos testes em `BacktrackingSolverTest`, que percorrem todas as células da solução retornada e invocam `SudokuValidator.isValid` célula a célula,

após temporariamente zerá-las para evitar falso conflito da célula consigo mesma.

O algoritmo termina em tempo finito, uma vez que o número de células é estritamente limitado a N^2 , cada célula aceita no máximo N valores distintos, e a árvore de busca gerada pelo mecanismo de retrocesso é finita e acíclica.

A aciclicidade decorre do fato de que cada nível de recursão corresponde a uma célula vazia distinta: ao descer na recursão, o algoritmo sempre avança para uma célula ainda não preenchida, e ao retroceder, desfaz exatamente a última atribuição e tenta o próximo candidato na mesma célula antes de subir um nível. Dessa forma, nenhum ramo da árvore é explorado mais de uma vez, e a profundidade máxima da pilha de recursão corresponde ao número de células vazias no tabuleiro inicial, que é sempre menor que N^2 , impedindo qualquer possibilidade de recursão infinita.

3.2.2. Análise de Desempenho

3.2.2.1. Espacial

A complexidade espacial do Backtracking é composta por dois componentes principais: o espaço ocupado pela representação do tabuleiro e o espaço consumido pela pilha de chamadas recursivas.

O tabuleiro é representado pela classe `SudokuBoard` como uma matriz `int[N][N]`. Diferentemente do cenário de dimensão fixa, em que $N=9$ permanece constante em todas as execuções e esse termo pode ser tratado como $O(1)$, neste trabalho N varia entre as instâncias comparadas (4, 9, 16 e 25), de modo que o espaço ocupado pelo modelo de dados deve ser expresso como $O(N^2)$. A matriz booleana auxiliar `boolean[N][N]`, responsável por registrar as células fixas, acrescenta outro bloco de mesma ordem, sem alterar a complexidade total do modelo de dados.

O componente adicional na análise espacial é a pilha de recursão. Cada

chamada recursiva a backtrack empilha um novo frame de ativação contendo as variáveis locais de controle dos laços: linha, coluna e número, além do endereço de retorno e da referência ao objeto Metrics. A profundidade máxima da pilha é igual ao número de células vazias no tabuleiro inicial, denotado por n , que varia de zero, no caso de um tabuleiro já resolvido, até um valor próximo de N^2 nas instâncias mais difíceis, por exemplo, até aproximadamente 64 no Sudoku 9×9, mas podendo se aproximar de 624 no Sudoku 25×25. A complexidade espacial da pilha é, portanto, $O(n)$, com $n \leq N^2 - 1$.

A complexidade espacial total do Backtracking é, assim, $O(n + N^2)$, combinando a pilha de recursão com o custo de representação do tabuleiro termo que, embora desprezível quando N é fixo, passa a ser relevante na comparação entre instâncias de tamanhos distintos.

Um aspecto crítico para essa complexidade favorável é o fato de o algoritmo realizar todas as atribuições e desfazimentos *in-place* na mesma instância de SudokuBoard, sem criar cópias do tabuleiro a cada nível de recursão. Essa decisão de implementação é fundamental, pois se uma cópia fosse criada a cada chamada recursiva, o consumo de memória escalaria para $O(n \times N^2)$, o que se tornaria particularmente custoso nas instâncias de maior dimensão, como o Sudoku 25×25.

3.2.2.2. Temporal

A análise da complexidade temporal do Backtracking para o Sudoku exige distinguir entre o pior caso teórico e o comportamento esperado na prática, considerando o papel fundamental das restrições na poda do espaço de busca, além do próprio crescimento de N .

No pior caso teórico, o algoritmo explora todas as combinações possíveis de valores para as n células vazias antes de encontrar uma solução ou declarar insolubilidade. Com cada célula podendo receber até N valores, o pior caso é $O(N^n)$. Para o Sudoku 9×9 ($n = 81$), isso resulta em $O(9^{81}) \approx 10^{77}$; para o Sudoku 25×25 (n

até 624), o limite teórico é astronomicamente superior, o que evidencia por que a viabilidade prática do algoritmo depende inteiramente da eficácia da poda por restrições à medida que N cresce.

Na prática, contudo, esse limite superior é muito pessimista e nunca se materializa, pois as verificações de restrição eliminam a imensa maioria dos ramos logo nas primeiras atribuições, e a estrutura do jogo impõe uma densidade de restrições suficiente para tornar a busca tratável.

O mecanismo de poda reduz drasticamente o número de nós efetivamente explorados. Cada chamada a `SudokuValidator.isValid` verifica no máximo $3N$ células (N da linha, N da coluna e N da subgrade), com custo $O(N)$, diferentemente do cenário de dimensão fixa, em que esse custo era tratado como $O(27) = O(1)$ por N ser sempre 9. A quantidade e a distribuição das pistas influenciam o espaço de busca. Em geral, instâncias com maior quantidade de pistas tendem a impor mais restrições, embora a dificuldade também dependa da configuração dessas pistas. Para as quatro dimensões (4×4 , 9×9 , 16×16 e 25×25) e os três níveis de dificuldade utilizados neste trabalho, o número empírico de nós visitados é coletado pela classe `Metrics` e comparado diretamente com o Branch and Bound na análise experimental, permitindo observar como cada técnica escala com N .

Cada nó visitado executa um laço de varredura linear sobre as N^2 células para localizar a próxima célula vazia, com custo $O(N^2)$ e não mais $O(1)$, já que N deixa de ser constante. Em seguida, realiza até N verificações de validade, cada uma com custo $O(N)$, totalizando $O(N^2)$ para o conjunto de tentativas naquela célula. O custo total por nó é, portanto, $O(N^2)$, e o custo total da execução é proporcional ao produto entre o número de nós visitados e N^2 , sendo essa relação a mais representativa do impacto conjunto da dificuldade da instância e do tamanho do tabuleiro sobre o desempenho real do algoritmo.

O Backtracking não possui nenhum mecanismo de poda antecipada além da verificação de validade a cada atribuição individual, o que faz com que ramos inviáveis só sejam descartados quando uma contradição é efetivamente encontrada,

e não antes. Essa limitação motiva diretamente a estratégia adotada pelo Branch and Bound, que incorpora uma função de bound capaz de detectar inviabilidade antes de descer mais um nível na recursão e cuja vantagem relativa, como discutido a seguir, também é afetada pelo crescimento de N .

3.3. Branch and Bound

O Branch and Bound é uma técnica algorítmica de busca sistematizada que combina a exploração recursiva da árvore de decisão, característica do Backtracking, com mecanismos de poda antecipada de ramos inviáveis. O nome da técnica reflete suas duas operações fundamentais: o *branching*, que consiste em ramificar o espaço de busca ao atribuir um valor a uma variável e gerar subproblemas menores, e o *bounding*, que consiste em avaliar, antes de explorar um ramo, se ele ainda tem condições de levar a uma solução válida, descartando-o precocemente caso contrário.

No contexto do Sudoku, a diferença central em relação ao Backtracking clássico não está na ordem de seleção das células, que permanece linear em ambos os algoritmos, mas na presença de uma etapa de avaliação prospectiva após cada atribuição. Essa etapa antecipa se o ramo atual da árvore de busca ainda tem condições de levar a uma solução válida, permitindo abandoná-lo antes mesmo de se aprofundar nele.

A implementação está centralizada na classe BranchAndBoundSolver, que percorre as células vazias em ordem linear, de forma idêntica ao BacktrackingSolver, delegando exclusivamente à função de bound a responsabilidade de diferenciar o comportamento do algoritmo.

A função de bound é implementada pelo método calcularBound, invocado após cada atribuição de valor a uma célula. O método percorre todas as células vazias restantes do tabuleiro e conta quantas delas ficaram sem nenhum candidato válido após a atribuição corrente. Se esse contador for maior que zero, o ramo é

podado imediatamente: o valor atribuído é desfeito, um backtrack é registrado nas métricas e o algoritmo passa para o próximo candidato sem descer na recursão. Apenas quando o bound retorna zero, indicando que nenhuma célula vazia ficou inviável, o algoritmo considera que o ramo é promissor e desce para o próximo nível. Sem essa verificação antecipada, os dois algoritmos seriam equivalentes em comportamento.

Outra distinção relevante está na forma como os candidatos são obtidos. O método obterCandidatos do BranchAndBoundSolver constrói um array de até $N+1$ posições, onde o índice 0 guarda a contagem de candidatos válidos e os índices 1 a N armazenam os valores candidatos dimensão que, assim como o tabuleiro, varia com N , e não é mais um bloco de tamanho fixo como no cenário de dimensão única.

3.3.1. Análise de Corretude

A corretude do Branch and Bound pode ser demonstrada pelas mesmas três propriedades examinadas para o Backtracking: completude, validade e terminação, com atenção especial ao impacto da função de bound sobre cada uma delas.

O algoritmo é completo, ou seja, se existe uma solução para o jogo, ela será encontrada. A prova de completude é análoga à do Backtracking: o espaço de busca é finito, a varredura é exaustiva dentro de cada ramificação, e o retrocesso garante que todas as alternativas sejam exploradas antes de declarar insolubilidade. A função de bound não compromete a completude, pois poda exclusivamente ramos em que alguma célula vazia ficou sem candidatos válidos, situação que, por definição, não pode levar a nenhuma solução válida.

A correção do laço de iteração sobre os candidatos pode ser estabelecida por meio de uma invariante de laço: ao início de cada iteração, o tabuleiro encontra-se em um estado parcialmente preenchido que é consistente com todas as restrições

do Sudoku. Essa invariante é válida na inicialização, pois o tabuleiro de entrada é uma instância bem formada cujas pistas não se contradizem entre si.

É mantida a cada iteração, pois apenas candidatos previamente validados pelo método `obterCandidatos` são atribuídos ao tabuleiro: o método invoca `SudokuValidator.isValid` para cada valor de 1 a N e inclui na lista somente os valores que não violam nenhuma das três restrições. Caso a lista de candidatos esteja vazia, nenhuma atribuição é realizada, o algoritmo retrocede e o estado consistente anterior é restaurado. É satisfeita na terminação, pois quando o laço encerra com retorno `true`, todas as células foram preenchidas e a invariante de consistência global está integralmente preservada.

Quando o algoritmo retorna `true`, o tabuleiro está completo e todas as restrições são satisfeitas, sendo essa propriedade verificada formalmente pelos testes em `BranchAndBoundSolverTest` e `SolverComparisonTest`.

O algoritmo termina em tempo finito pelos mesmos argumentos do Backtracking. O número de células é limitado a N^2 , cada célula aceita no máximo N valores e a árvore de busca gerada pelo mecanismo de retrocesso é finita e acíclica. A profundidade máxima da pilha de recursão continua sendo $O(n)$, onde n é o número de células vazias no tabuleiro inicial ($n \leq N^2 - 1$).

O Branch and Bound possui um mecanismo adicional de poda que pode reduzir significativamente o número de estados explorados. Entretanto, essa redução não garante menor tempo de execução, pois a avaliação do bound possui custo adicional.

3.3.2. Análise de Desempenho

3.3.2.1. Espacial

A complexidade espacial do Branch and Bound compartilha a mesma estrutura de análise do Backtracking, com um componente adicional introduzido pela

função de bound.

O espaço ocupado pelo modelo de dados é idêntico ao do Backtracking: a matriz $\text{int}[N][N]$ de SudokuBoard e a matriz booleana auxiliar $\text{boolean}[N][N]$ ocupam espaço $O(N^2)$, e não mais $O(1)$, pelo mesmo motivo já discutido, N varia entre as instâncias comparadas. O algoritmo também opera *in-place*, sem criar cópias do tabuleiro a cada nível de recursão.

A pilha de recursão continua sendo um dos componentes dominantes, com profundidade máxima igual ao número de células vazias n e complexidade $O(n)$.

O componente adicional introduzido pelo Branch and Bound é o array de candidatos gerado a cada chamada recursiva por obterCandidatos: um array $\text{int}[N+1]$ alocado localmente. Diferentemente do cenário de dimensão fixa, em que esse array tinha tamanho constante (10, para $N=9$) e portanto não influenciava a ordem de complexidade, aqui seu tamanho é $O(N)$, de modo que o espaço total ocupado simultaneamente pela pilha é $O(n \times N)$.

A complexidade espacial total do Branch and Bound é, portanto, $O(n \times N + N^2)$, superior à do Backtracking simples ($O(n + N^2)$) justamente pelo custo adicional do array de candidatos por frame, um efeito que só se torna perceptível quando N passa a variar entre as instâncias analisadas.

3.3.2.2. Temporal

A análise temporal do Branch and Bound envolve o custo adicional da função de bound a cada nível de recursão, compensado pela redução no número total de nós visitados proporcionada pela poda antecipada. O limite superior teórico do Branch and Bound continua sendo $O(N^n)$, o mesmo do Backtracking, pois no pior caso absoluto a função de bound não consegue reduzir o espaço de busca de forma assintoticamente significativa.

A cada chamada recursiva, a seleção de célula segue ordem linear com custo $O(N^2)$, idêntico ao Backtracking. Após cada atribuição, o método calcularBound percorre as n células vazias restantes e, para cada uma, verifica a existência de

candidatos válidos com custo $O(N)$ totalizando $O(n \times N)$ por chamada a `calcularBound`. Diferentemente do cenário de dimensão fixa, em que esse custo era simplificado para $O(n \times 9) = O(n)$ (pois $N=9$ era constante), aqui o termo N não pode mais ser absorvido, e o overhead da função de bound é $O(n \times N)$ por nó visitado.

A vantagem do Branch and Bound se manifesta quando o número de nós que a função de bound evita explorar é suficientemente grande para compensar esse custo adicional por nó, custo que cresce tanto com a dificuldade da instância (via n) quanto com a dimensão do tabuleiro (via N).

Sejam V_{bt} o número de nós visitados pelo Backtracking e V_{bb} o número de nós visitados pelo Branch and Bound para uma mesma instância. O custo total do Backtracking é $O(V_{bt} \times N^2)$ e o do Branch and Bound é $O(V_{bb} \times n \times N)$. O Branch and Bound é mais eficiente quando essa redução no número de nós visitados compensa o custo adicional por nó sendo uma relação que, com a introdução da variação de N , passa a depender não apenas da dificuldade da instância, mas também de sua dimensão, sendo esse justamente um dos pontos centrais de investigação da escalabilidade proposta neste trabalho.

Para instâncias fáceis e de menor dimensão (como 4×4), onde ambos os algoritmos visitam poucos nós, a diferença tende a ser pequena, podendo o overhead do bound até tornar o Branch and Bound ligeiramente mais lento. Para instâncias difíceis e de maior dimensão (como 25×25), a redução em V_{bb} tende a dominar, e o Branch and Bound deve apresentar vantagem mais expressiva, comportamento a ser quantificado empiricamente na seção de resultados, por meio das métricas coletadas pela classe `Metrics` em cada uma das quatro dimensões testadas.

3.4. Programação Dinâmica

A Programação Dinâmica é uma técnica algorítmica que resolve problemas complexos decompondo-os em subproblemas menores e armazenando os

resultados desses subproblemas para evitar recomputação redundante. A abordagem implementada não corresponde a uma formulação clássica de programação dinâmica baseada em uma recorrência com subproblemas independentes. Trata-se de uma busca com memoização de estados malsucedidos.

No contexto do Sudoku, a diferença central em relação ao Backtracking clássico reside no tratamento de estados parciais já explorados. Enquanto o Backtracking pode revisitar configurações intermediárias do tabuleiro por caminhos distintos na árvore de busca, a Programação Dinâmica serializa cada estado parcial em uma chave canônica e o registra como estado morto caso a exploração daquele estado não tenha levado a nenhuma solução. Nas ocorrências subsequentes do mesmo estado, o algoritmo reconhece imediatamente a inviabilidade e retrocede sem repetir a expansão.

A implementação está centralizada na classe `DynamicProgrammingSolver`, que mantém um conjunto de estados mortos compartilhado entre todas as chamadas recursivas. A seleção da próxima célula segue ordem linear, de forma idêntica ao `BacktrackingSolver`, isolando a memoização como único mecanismo diferencial em relação à busca simples.

A função de memoização é implementada pela serialização do tabuleiro em uma chave de **N^2 caracteres**, invocada no início de cada chamada recursiva e não mais fixa em 81 caracteres, já que N varia entre as instâncias testadas. O método percorre todas as N^2 células do tabuleiro, concatena seus valores em uma string e consulta o conjunto de estados mortos. Se a chave já estiver presente, o ramo é podado imediatamente. Apenas quando a chave é nova o algoritmo prossegue: seleciona a próxima célula vazia, tenta os valores de 1 a N , valida cada tentativa com `SudokuValidator.isValid`, avança recursivamente quando a atribuição é consistente e desfaz a jogada quando necessário. Se nenhum valor funcionar para aquele estado, a chave é inserida no conjunto de estados mortos antes do retrocesso.

Assim como no Branch and Bound, o modo de visualização passo a passo

pode ser ativado via parâmetro booleano no construtor, sendo desativado nos experimentos em lote com `new DynamicProgrammingSolver(false)`.

3.4.1. Análise de Corretude

A corretude da Programação Dinâmica pode ser demonstrada pelas mesmas três propriedades examinadas para o Backtracking e o Branch and Bound: completude, validade e terminação, com atenção especial ao impacto da memoização sobre cada uma delas.

O algoritmo é completo: a memoização registra exclusivamente estados para os quais já existe prova de insolubilidade, de modo que nenhum estado promissor é descartado. A prova de completude é análoga à do Backtracking, com o acréscimo de que a memoização apenas evita recomputação de estados já esgotados, sem introduzir viés de omissão.

A validade é estabelecida por uma invariante recursiva: em toda chamada, o tabuleiro parcial respeita as regras de unicidade em linha, coluna e subgrade, condição garantida pela verificação `SudokuValidator.isValid` sobre cada valor de 1 a N antes de qualquer atribuição. A memoização não interfere nessa invariante: atua apenas no controle de fluxo da busca, descartando estados inviáveis na entrada da chamada, antes de qualquer tentativa de atribuição. Essa propriedade é verificada formalmente pelos testes em `DynamicProgrammingSolverTest` e `SolverComparisonTest`.

O algoritmo termina em tempo finito pelos mesmos argumentos do Backtracking. O número de células é limitado a N^2 , cada célula aceita no máximo N valores, e a árvore de busca gerada é finita e acíclica. Adicionalmente, a memoização reforça a terminação ao impedir que o algoritmo reexpanda estados já visitados: cada estado é expandido no máximo uma vez.

3.4.2. Análise de Desempenho

3.4.2.1. Espacial

A complexidade espacial da Programação Dinâmica compartilha a estrutura de análise do Backtracking, com um componente adicional introduzido pela estrutura de memoização.

O espaço ocupado pelo modelo de dados é o mesmo dos demais algoritmos: $\text{int}[N][N]$ e $\text{boolean}[N][N]$, ambos $O(N^2)$. A pilha de recursão mantém complexidade $O(n)$, com $n \leq N^2 - 1$.

O componente diferencial da Programação Dinâmica é o conjunto de estados mortos: uma estrutura $\text{HashSet}\langle\text{String}\rangle$ que armazena as chaves dos estados já provados insolúveis. Diferentemente do cenário de dimensão fixa, em que cada chave tinha 81 caracteres um tamanho constante e, portanto, ignorável na análise, aqui cada chave possui N^2 caracteres, de modo que cada entrada do conjunto ocupa espaço $O(N^2)$, e não mais $O(1)$. Sendo M o número de estados mortos acumulados, o espaço ocupado pelo conjunto é $O(M \times N^2)$.

A complexidade espacial total da Programação Dinâmica é, portanto, $O(n + M \times N^2)$ uma revisão relevante em relação ao cenário de dimensão fixa, em que a expressão $O(n + M)$ já capturava corretamente o comportamento, pois o custo por chave (81) era constante. Em instâncias de dimensão maior (16×16, 25×25), o crescimento de N^2 penaliza diretamente o custo por estado memoizado, tornando a Programação Dinâmica potencialmente mais sensível ao aumento de N do que o Branch and Bound, cuja complexidade espacial é $O(n \times N + N^2)$.

3.4.2.2. Temporal

A análise temporal da Programação Dinâmica envolve dois fatores que não estão presentes no Backtracking simples: o custo adicional da serialização do estado e da consulta ao conjunto de estados mortos a cada nível de recursão, e a redução no número total de nós visitados proporcionada pelo reconhecimento imediato de estados inviáveis já explorados.

O limite superior teórico continua sendo $O(N^n)$, o mesmo do Backtracking e do Branch and Bound.

A cada chamada recursiva, antes de qualquer atribuição, o algoritmo serializa o tabuleiro em uma string de N^2 caracteres, com custo **$O(N^2)$** e não mais $O(81) = O(1)$, já que N varia entre as instâncias testadas. A consulta ao HashSet tem custo esperado $O(1)$ por operação de hash, mas o próprio cálculo do hash e a comparação de igualdade de strings de comprimento N^2 custam $O(N^2)$, de modo que o custo efetivo da consulta/inserção também deve ser expresso como **$O(N^2)$** , e não como constante.

O overhead total por nó visitado na Programação Dinâmica é, portanto, **$O(N^2)$** , decorrente da serialização e da consulta/inserção no conjunto de estados mortos. Comparado ao Branch and Bound, cujo overhead por nó é $O(n \times N)$, a relação entre as duas técnicas deixa de ser simplesmente "DP tem overhead constante, B&B tem overhead $O(n)$ " como seria válido apenas quando N está fixo e passa a depender da relação entre N^2 e $n \times N$, ou seja, entre N e n : para instâncias em que n é grande em relação a N (tabuleiros com poucas pistas), o overhead do Branch and Bound tende a crescer mais rapidamente que o da Programação Dinâmica; para tabuleiros de dimensão muito maior (N grande) com poucas células vazias, o custo de serialização da Programação Dinâmica pode se tornar comparativamente mais significativo.

Sejam V_{bt} , V_{bb} e V_{dp} o número de nós visitados por Backtracking, Branch and Bound e Programação Dinâmica, respectivamente, para uma mesma instância. O custo total é $O(V_{bt} \times N^2)$ para o Backtracking, $O(V_{bb} \times n \times N)$ para o

Branch and Bound e $O(V_{dp} \times N^2)$ para a Programação Dinâmica. A comparação entre as três técnicas, portanto, passa a depender explicitamente de N , e não apenas do número de nós visitados o que reforça o valor do experimento de escalabilidade proposto: observar como essa relação se desloca à medida que o tabuleiro cresce de 4×4 para 25×25 , é quantificado empiricamente na seção de resultados por meio das métricas coletadas pela classe Metrics em cada dimensão e nível de dificuldade avaliados.

4. METODOLOGIA

Nesta seção são apresentados os procedimentos adotados para o desenvolvimento e avaliação do sistema de resolução de Sudoku. São descritos o ambiente computacional utilizado, a linguagem de programação e as principais tecnologias empregadas, as estruturas de dados utilizadas na representação do problema e na implementação dos algoritmos, bem como os critérios definidos para a realização dos experimentos e comparação de desempenho.

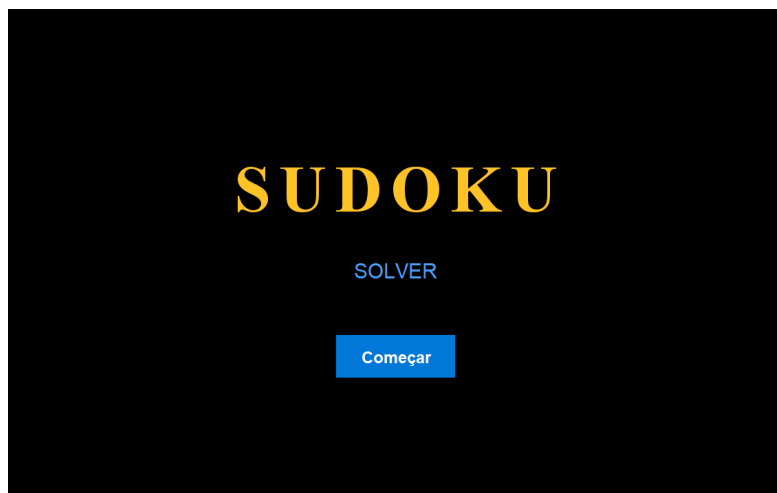
4.1 Ambiente Computacional

Os experimentos foram realizados em um computador equipado com processador Intel Core 5, 8 GB de memória RAM e sistema operacional Windows 11 de 64 bits. Para a compilação e execução do projeto, foi utilizado o Java Development Kit (JDK) 23 ou superior, juntamente com o Apache Maven para gerenciamento das dependências, compilação e organização do projeto.

Todos os algoritmos foram executados sob as mesmas condições computacionais e utilizando as mesmas instâncias de Sudoku, buscando garantir uma comparação consistente entre as diferentes abordagens. Durante os experimentos, foram coletadas métricas de desempenho, incluindo tempo de execução, quantidade de nós visitados, número de backtracks, quantidade de podas, profundidade máxima, chamadas recursivas e consumo de memória.

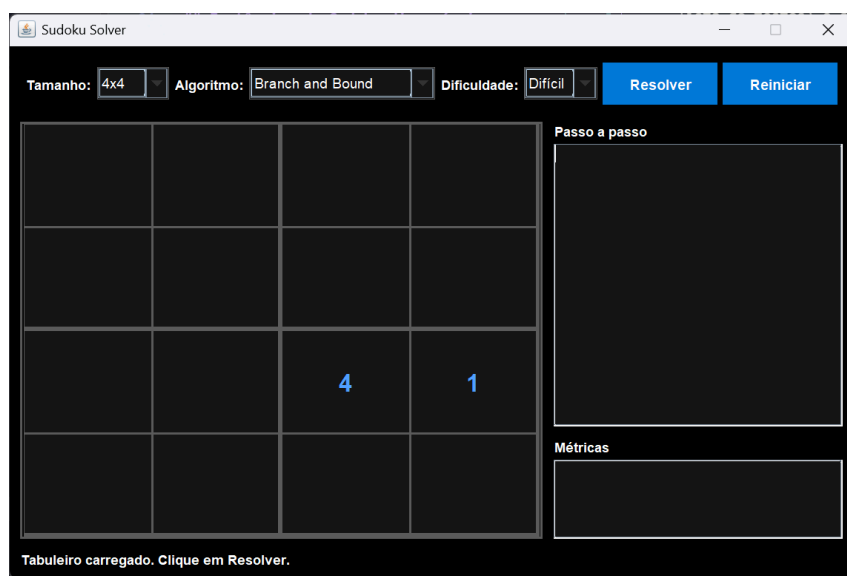
O sistema também possui uma interface gráfica desenvolvida em Java Swing, que permite selecionar o tamanho do tabuleiro, o algoritmo e o nível de dificuldade, além de visualizar a solução e as métricas obtidas durante a execução.

Figura 1 - Interface SUDOKU.



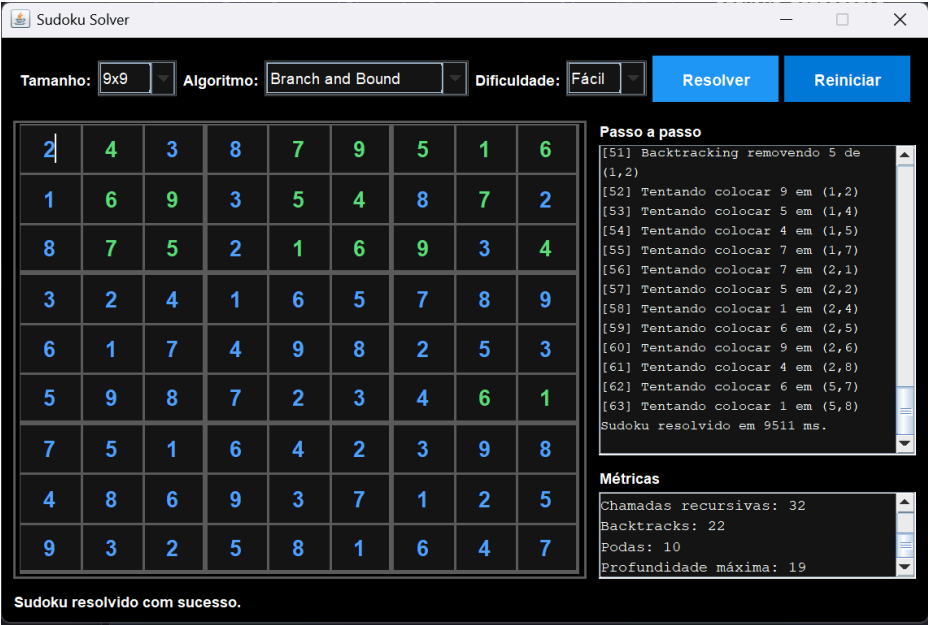
Fonte: Autoria própria (2026).

Figura 2 - Interface resolver SUDOKU 4x4, branch and bound, difícil.



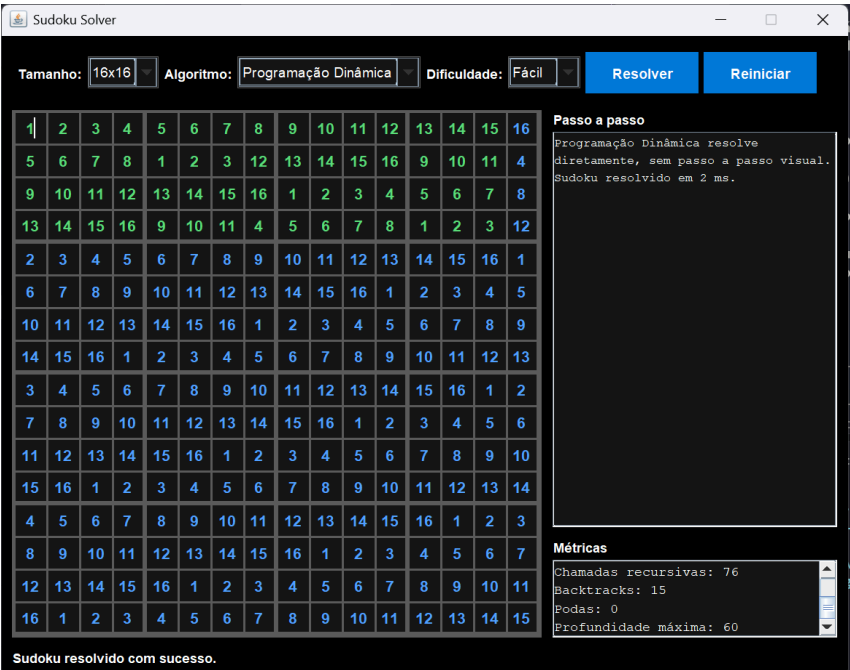
Fonte: Autoria própria (2026).

Figura 3 - Interface resolver SUDOKU 9x9, branch and bound, fácil.



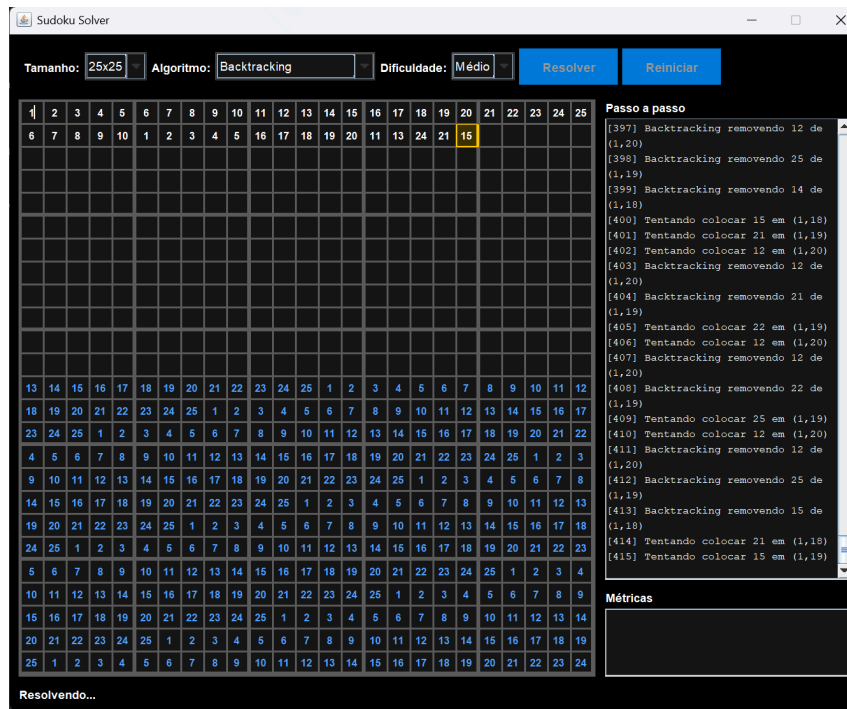
Fonte: Autoria própria (2026).

Figura 4 - Interface resolver SUDOKU 16x16, Programação dinâmica, fácil.



Fonte: Autoria própria (2026).

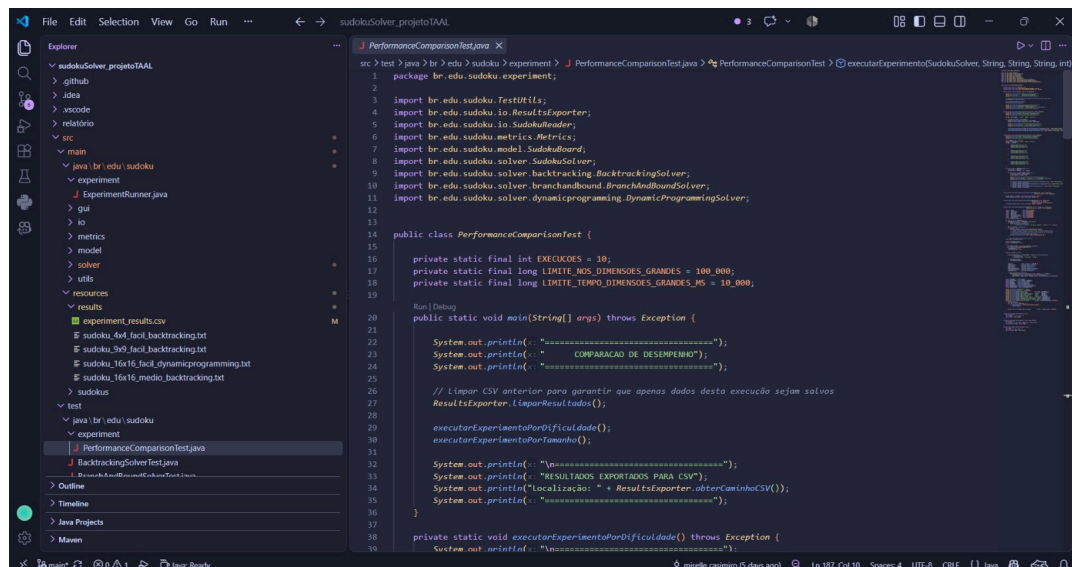
Figura 5 - Interface resolver SUDOKU 25x25, Backtracking, Médio.



Fonte: Autoria própria (2026).

A estrutura geral do projeto é organizada da seguinte forma:

Figura 6 - Estrutura do Projeto IDE VSCODE



Nota: Captura de tela realizada na plataforma Vscode. Fonte: Autoria própria (2026).

O projeto, incluindo o código-fonte, arquivos de entrada dos tabuleiros, configurações necessárias e demais recursos utilizados nos experimentos, encontra-se disponível em um repositório Git, permitindo a consulta, reprodução e análise dos procedimentos realizados, aqui: <[REPOSITÓRIO NO GIT](#)>

4.1.1 Configuração dos Experimentos

Os experimentos de desempenho são executados pela classe *PerformanceComparisonTest*, responsável por realizar automaticamente as combinações entre os algoritmos, tamanhos de tabuleiro e níveis de dificuldade definidos para o estudo.

Cada configuração é executada 10 vezes, permitindo reduzir a influência de variações momentâneas do ambiente de execução e obter medidas estatísticas mais representativas.

Para os tabuleiros de maior dimensão, especialmente 16×16 e 25×25, são utilizados limites de tempo e de quantidade de nós visitados. Essa medida é necessária devido ao crescimento acentuado do espaço de busca do Sudoku à medida que sua dimensão aumenta.

Quando uma execução ultrapassa um desses limites, ela é registrada como **não concluída**, em vez de ser descartada. Essa informação é importante para a análise de escalabilidade, pois permite identificar experimentalmente o ponto em que determinada abordagem deixa de ser viável dentro das condições estabelecidas.

4.2 Linguagem Utilizada

O sistema foi desenvolvido em **Java**, linguagem escolhida por oferecer suporte à programação orientada a objetos, recursão, interfaces e estruturas de dados necessárias à implementação dos algoritmos estudados.

A arquitetura utiliza uma interface comum denominada SudokuSolver, que define o contrato de resolução utilizado pelos três algoritmos: `boolean solve(SudokuBoard board, Metrics metrics);`

Essa abordagem permite que os diferentes algoritmos sejam tratados de maneira uniforme pelo restante do sistema. Dessa forma, o componente responsável pela execução dos experimentos pode selecionar o algoritmo sem depender diretamente dos detalhes de sua implementação.

Foram implementadas três técnicas de projeto de algoritmos:

- **Backtracking;**
- **Branch and Bound;**
- **Programação Dinâmica**, utilizando memoização.

Além dos algoritmos de resolução, o Java foi utilizado no desenvolvimento da interface gráfica com a biblioteca Swing e na implementação dos testes automatizados com JUnit 5. O Apache Maven foi utilizado para gerenciamento das dependências, compilação e execução dos testes.

4.3 Estrutura de dados

A principal estrutura de dados utilizada pelo sistema é a classe SudokuBoard, responsável por representar o estado atual do tabuleiro.

Diferentemente da implementação inicial limitada ao Sudoku 9×9, a versão atual foi generalizada para trabalhar com diferentes dimensões. A classe recebe o tamanho do tabuleiro por meio do atributo: `private int size;` e utiliza duas matrizes bidimensionais: `private int[][] board;` `private boolean[][] fixed;`

A matriz `board` armazena os valores atualmente presentes no Sudoku. O valor 0 representa uma célula vazia, enquanto os valores de 1 até N representam células preenchidas.

A matriz `fixed` registra quais células continham valores originalmente fornecidos no arquivo de entrada. Dessa maneira, o sistema consegue diferenciar as pistas originais dos valores posteriormente inseridos pelo algoritmo.

A representação permite trabalhar com os quatro tamanhos utilizados nos experimentos:

TAMANHO	SUBGRADE	VALORES POSSÍVEIS
4x4	2x2	1 a 4
9x9	3x3	1 a 9
16x16	4x4	1 a 16
25x25	5x5	1 a 25

Tabela 1 - Representação dos tamanhos do sudoku.

A relação entre o tamanho do tabuleiro e o tamanho da subgrade é determinada pela raiz quadrada de N . Essa característica permite que a mesma estrutura de dados seja utilizada para todas as dimensões analisadas.

4.3.1. Estrutura utilizada pelo Backtracking

O `BacktrackingSolver` utiliza a própria matriz do objeto `SudokuBoard` durante a busca. As atribuições são realizadas diretamente no tabuleiro e, quando um ramo não conduz a uma solução, o valor inserido é removido, restaurando a célula para 0.

Essa estratégia caracteriza uma operação **in-place**, evitando a criação de uma nova cópia do tabuleiro a cada chamada recursiva. A busca utiliza a própria pilha de execução para armazenar os estados necessários à recursão.

4.3.2. Estruturas utilizadas pelo Branch and Bound

O `BranchAndBoundSolver` também utiliza o `SudokuBoard` de maneira in-place. Além disso, o algoritmo utiliza estruturas auxiliares para armazenar os candidatos possíveis para uma célula. O método `obterCandidatos` considera os

valores de 1 até N, verificando quais podem ser inseridos sem violar as restrições do Sudoku.

A principal estrutura adicional está relacionada ao mecanismo de *bound*. Após uma atribuição, o método `calcularBound` verifica as células vazias restantes e identifica situações nas quais alguma delas não possui nenhum candidato válido. Quando isso ocorre, o ramo é considerado inviável e é podado antes que uma nova chamada recursiva seja realizada.

4.3.3. Estruturas utilizadas pela Programação Dinâmica

A implementação da Programação Dinâmica utiliza **memoização** para evitar a exploração repetida de estados já identificados como insolúveis. Para isso, o `DynamicProgrammingSolver` utiliza uma estrutura: `HashSet<String> estadosMortos`;

Cada estado do tabuleiro é transformado em uma representação textual contendo os valores das suas células. Como a representação depende do tamanho do tabuleiro, o estado possui N^2 posições.

Quando um estado é completamente explorado sem produzir uma solução, sua representação é adicionada ao conjunto `estadosMortos`. Caso o mesmo estado seja encontrado novamente durante a busca, ele é descartado imediatamente, evitando a repetição da mesma exploração.

Essa estrutura é particularmente relevante para a análise de escalabilidade, pois o custo de armazenamento dos estados aumenta conforme o tamanho do tabuleiro cresce.

4.3.4. Estruturas auxiliares

Além das estruturas diretamente relacionadas aos algoritmos, o sistema possui componentes auxiliares. A classe `SudokuValidator` é responsável por verificar se determinado valor pode ser inserido em uma célula sem violar as restrições da linha, coluna e subgrade correspondente.

A classe Metrics armazena as informações coletadas durante cada execução. Entre as métricas utilizadas estão:

- tempo de execução;
- memória utilizada;
- nós visitados;
- chamadas recursivas;
- número de *backtracks*;
- número de podas;
- profundidade máxima da busca.

Os resultados dos experimentos são armazenados pela classe ResultsExporter em um arquivo CSV, permitindo posteriormente a geração de tabelas e gráficos para análise.

4.4 Critérios Experimentais

A avaliação experimental foi planejada com o objetivo de comparar o comportamento dos três algoritmos diante do aumento do tamanho e da dificuldade dos tabuleiros. Foram consideradas quatro dimensões de Sudoku: 4×4; 9×9; 16×16; 25×25. E para cada dimensão foram utilizadas três categorias de dificuldade: fácil; médio; difícil. Dessa forma, o experimento considera diferentes combinações entre tamanho do problema, dificuldade da instância e técnica algorítmica.

4.4.1. Algoritmos avaliados

São comparados os seguintes algoritmos: Backtracking; Branch and Bound; e Programação Dinâmica com memoização.

Todos os algoritmos recebem o mesmo tabuleiro inicial para uma determinada instância, garantindo que a comparação seja realizada sobre condições

equivalentes.

Além disso, os três algoritmos utilizam a mesma estrutura de representação do tabuleiro e a mesma função de validação das restrições, permitindo concentrar a comparação nas estratégias de busca e poda de cada abordagem.

4.4.2. Repetição dos experimentos

Cada combinação entre algoritmo, tamanho e dificuldade é executada 10 vezes. Assim, para cada configuração são obtidas múltiplas medições, permitindo calcular estatísticas como:

- média do tempo de execução;
- desvio padrão do tempo;
- média de nós visitados;
- média de chamadas recursivas;
- média de *backtracks*;
- média de podas;
- profundidade máxima;
- memória utilizada.

Antes de cada execução, o tabuleiro é novamente carregado a partir do arquivo de entrada. Essa estratégia garante que uma execução não altere o estado utilizado pela próxima repetição. Da mesma forma, uma nova instância de *Metrics* é criada para cada execução, evitando que os contadores sejam acumulados entre os experimentos.

4.4.3. Métricas analisadas

A comparação dos algoritmos considera diferentes aspectos do comportamento da busca.

Métrica	Objetivo
Tempo de execução	Medir o custo temporal da resolução
Memória	Avaliar o consumo de recursos durante a execução
Nós visitados	Medir a quantidade de estados explorados
Chamadas recursivas	Quantificar a utilização da recursão
Backtracks	Medir quantos retrocessos ocorreram
Podas	Avaliar a quantidade de ramos descartados
Profundidade máxima	Observar a profundidade alcançada pela busca

Tabela 2 - Métricas e objetivos avaliadas no projeto

O tempo de execução é utilizado para verificar o custo efetivo de cada abordagem, enquanto a quantidade de nós visitados permite observar de maneira mais direta o tamanho do espaço de busca efetivamente explorado.

As métricas de *backtracks* e podas são especialmente relevantes para a comparação entre Backtracking e Branch and Bound, pois permitem observar o efeito dos mecanismos utilizados para reduzir a exploração da árvore de busca.

Já a quantidade de estados armazenados pela Programação Dinâmica permite avaliar o custo associado à estratégia de memoização.

4.4.4. Controle das condições de execução

Para minimizar interferências externas nas medições, os experimentos de

desempenho são realizados sem o modo de visualização passo a passo dos algoritmos. Embora o sistema possua uma funcionalidade de visualização para acompanhar a resolução, essa funcionalidade envolve operações de saída e atrasos deliberados, que alterariam significativamente o tempo medido.

Por esse motivo, nos experimentos automatizados são utilizadas instâncias dos solvers com a visualização desativada, garantindo que o tempo registrado represente o processamento do algoritmo e não o custo da apresentação visual.

Além disso, as execuções são repetidas dez vezes para cada configuração. A média das execuções é utilizada como medida principal, enquanto o desvio padrão permite observar a variação existente entre as repetições.

4.4.5. Critério de escalabilidade

O principal objetivo dos experimentos é analisar a escalabilidade dos algoritmos. Para isso, os resultados são observados conforme o tamanho do tabuleiro aumenta de: $4 \times 4 \rightarrow 9 \times 9 \rightarrow 16 \times 16 \rightarrow 25 \times 25$, mantendo-se as três categorias de dificuldade.

Essa abordagem permite investigar como o crescimento de N influencia:

- o número de estados explorados;
- o tempo de execução;
- o consumo de memória;
- a profundidade da busca;
- a quantidade de retrocessos;
- a quantidade de podas;
- a capacidade de conclusão dos algoritmos.

Para os tabuleiros 16×16 e 25×25 , são utilizados limites de tempo e de nós visitados. Caso o algoritmo atinja um desses limites antes de encontrar uma solução, a execução é registrada como **não concluída** no arquivo de resultados.

Essa situação não é tratada simplesmente como uma falha experimental. Pelo contrário, ela constitui uma informação relevante para a análise, pois demonstra empiricamente o ponto em que uma determinada abordagem deixa de apresentar viabilidade prática diante do crescimento do problema.

4.4.6. Registro dos resultados

Os resultados obtidos nas execuções são registrados no arquivo: **src/main/resources/results/experiment_results.csv**. O arquivo contém os dados necessários para a posterior construção das tabelas e gráficos apresentados na análise experimental.

Dessa forma, a metodologia estabelece uma separação entre a execução dos algoritmos, responsável pela coleta das métricas, e a análise dos resultados, realizada posteriormente a partir dos dados registrados.

5. EXPERIMENTOS E RESULTADOS

5.1. Instâncias de Testes

Os experimentos foram realizados com o objetivo de analisar o comportamento dos algoritmos Backtracking, Branch and Bound e Programação Dinâmica sob diferentes condições de entrada. Para isso, foram consideradas duas dimensões principais de análise: a dificuldade das instâncias e o tamanho do tabuleiro. A combinação dessas duas variáveis permite observar tanto o comportamento dos algoritmos diante de diferentes níveis de complexidade de uma mesma dimensão de problema quanto sua capacidade de lidar com o crescimento da entrada.

Foram utilizados quatro tamanhos de tabuleiro: 4×4 , 9×9 , 16×16 e 25×25 , sendo cada tamanho avaliado nas categorias de fácil, médio e difícil. Dessa forma, foram analisadas doze instâncias distintas para cada algoritmo.

Cada combinação de algoritmo, tamanho e dificuldade foi executada 10 vezes, permitindo calcular o tempo médio de execução e o desvio padrão, além das métricas relacionadas ao comportamento da busca.

Para os tabuleiros maiores, especialmente os de 25×25 , foi estabelecido um limite de 100.000 nós visitados por execução. Esse limite funciona como mecanismo de segurança experimental, evitando que instâncias com espaço de busca muito grande permaneçam em execução por tempo indefinido. Quando o limite é atingido antes da obtenção da solução, a execução é registrada como não concluída.

A análise dos resultados deve considerar, portanto, que tempo de execução, número de nós e número de backtracks representam aspectos diferentes do comportamento dos algoritmos. Um algoritmo pode visitar menos nós e ainda assim apresentar tempo maior devido ao custo adicional das operações realizadas em

cada nó. Da mesma forma, um algoritmo pode apresentar baixo tempo em uma instância pequena sem que isso signifique que sua estratégia seja necessariamente mais adequada para instâncias de maior escala.

5.2 Resultados por tamanho do tabuleiro

As tabelas desta seção apresentam os resultados para cada tamanho de tabuleiro dos três algoritmos que estão implementados e nos três níveis de dificuldade disponíveis.

Dificuldade	Algoritmo	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks	Podas	Memória usada (MB)*
Fácil (prof. 4)	Backtracking	0,020	0,022	5	0	0	10,96
	Branch and Bound	0,025	0,006	5	0	0	11,53
	Dynamic Programming	0,017	0,003	5	0	0	12,11
Médio (prof. 8)	Backtracking	0,013	0,004	9	0	0	12,68
	Branch and Bound	0,056	0,072	9	0	0	13,25
	Dynamic Programming	0,043	0,054	9	0	0	13,81
Difícil (prof. 14)	Backtracking	0,017	0,005	15	0	0	14,37
	Branch and Bound	0,020	0,002	15	0	0	14,91
	Dynamic Programming	0,019	0,006	15	0	0	15,50

Tabela 3 - Resultados do Sudoku 4x4 (fácil, médio e difícil)

Comparação dos resultados do tempo e desvio padrão em ms do tabuleiro 4x4

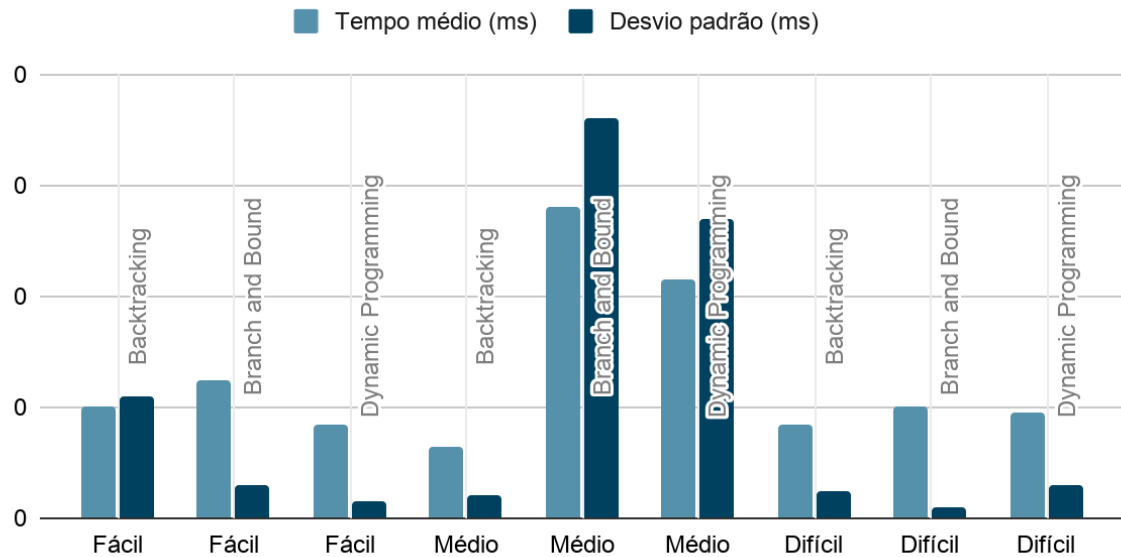


Gráfico 1 - Comparação do tempo e desvio padrão em ms dos resultados obtidos do tabuleiro 4x4.

Comparação do consumo de memória em MB dos resultados obtidos pelo tabuleiro 4x4

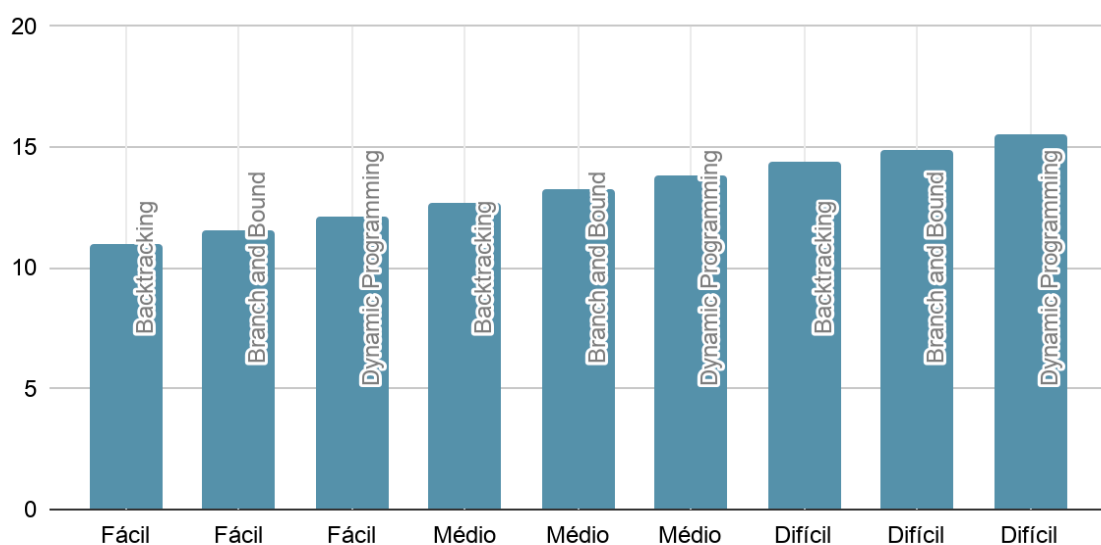


Gráfico 2 - Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 4x4.

Dificuldade	Algoritmo	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks	Podas	Memória usada (MB)*
Fácil (prof. 19)	Backtracking	0,055	0,011	70	50	0	16,12
	Branch and Bound	0,268	0,126	32	22	10	16,85
	Dynamic Programming	0,085	0,014	70	50	0	17,60
Médio (prof. 28)	Backtracking	0,076	0,040	86	57	0	18,47
	Branch and Bound	0,165	0,044	39	16	6	19,11
	Dynamic Programming	0,156	0,047	86	57	0	19,95
Difícil (prof. 38)	Backtracking	0,076	0,023	85	46	0	20,71
	Branch and Bound	0,367	0,143	52	24	11	21,43
	Dynamic Programming	0,160	0,043	85	46	0	22,27

Tabela 4 - Resultados do Sudoku 9x9 (fácil, médio e difícil)

Comparação tempo e desvio padrão em ms dos resultados obtidos do tabuleiro 9x9

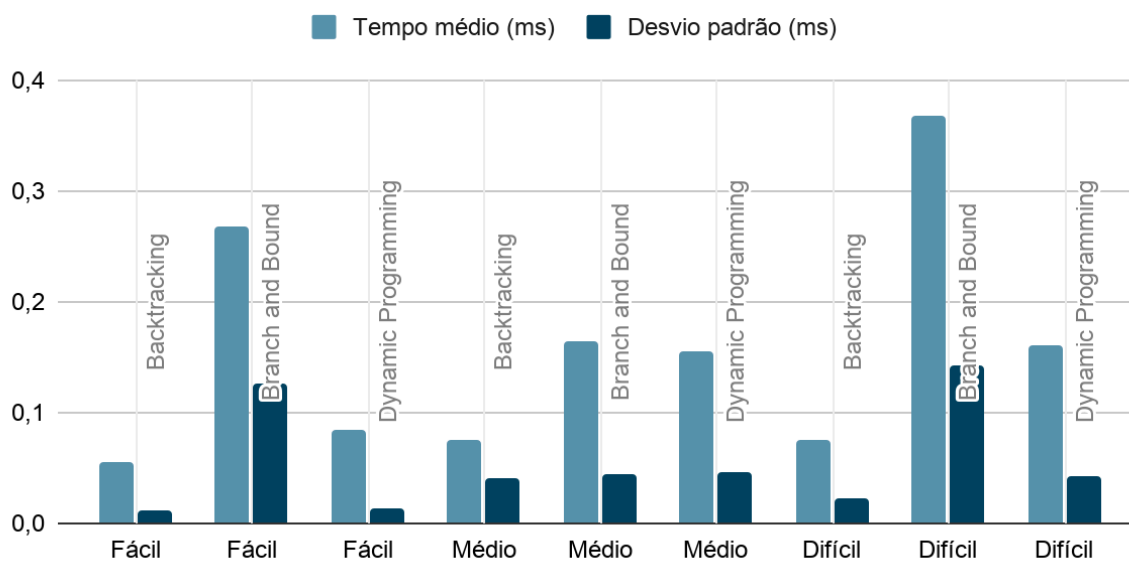


Gráfico 3 - Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 9x9.

Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 9x9

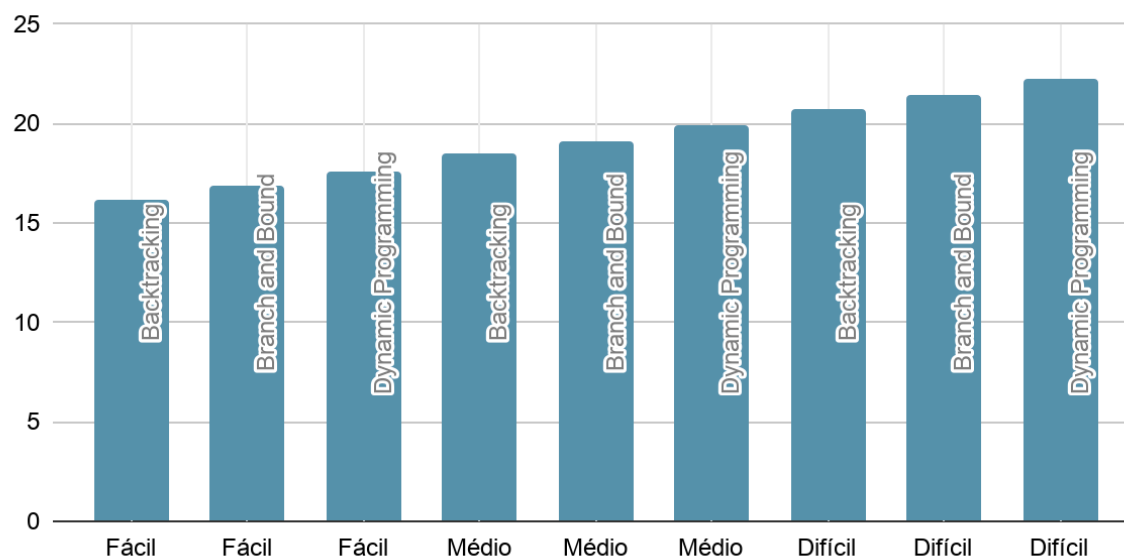


Gráfico 4 - Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 9x9.

Dificuldade	Algoritmo	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks	Podas	Memória usada (MB)*
Fácil (prof. 60)	Backtracking	0,261	0,163	76	15	0	23,20
	Branch and Bound	0,908	0,457	61	1	1	24,14
	Dynamic Programming	0,515	0,947	76	15	0	18,12†
Médio (prof. 128)	Backtracking	0,200	0,063	129	0	0	2,79†
	Branch and Bound	3,836	0,423	129	0	0	3,80
	Dynamic Programming	0,234	0,039	129	0	0	5,16
Difícil (prof. 192)	Backtracking	0,332	0,091	197	4	0	6,66
	Branch and Bound	9,353	1,833	193	4	4	7,74
	Dynamic Programming	0,368	0,083	197	4	0	9,38

Tabela 5 - Resultados do Sudoku 16x16 (fácil, médio e difícil)

Comparação do tempo e desvio padrão em ms dos resultados obtidos do tabuleiro 16x16

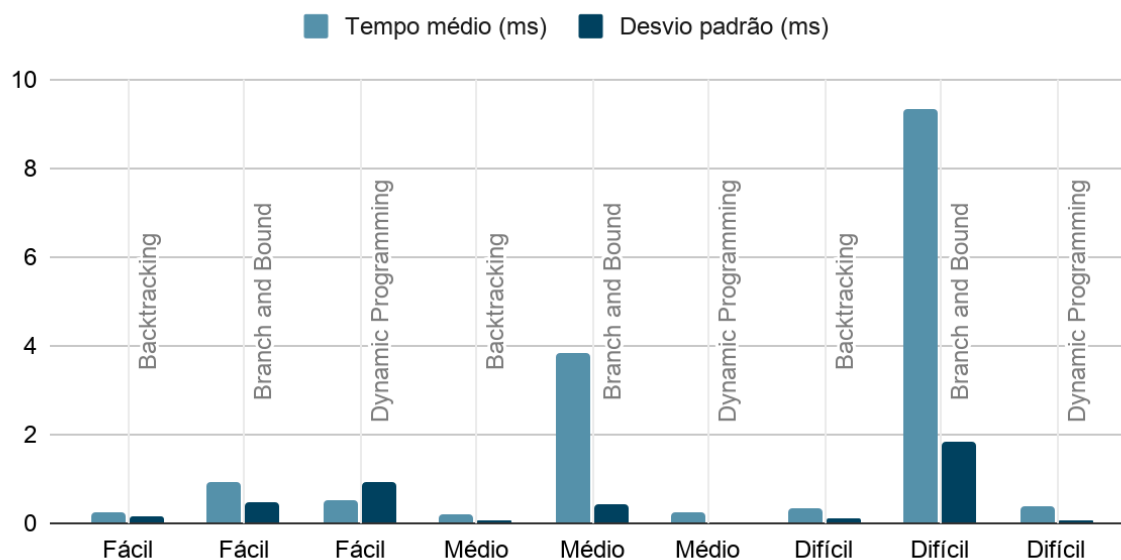


Gráfico 5 - Comparação do tempo e desvio padrão em ms dos resultados obtidos do tabuleiro

Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 16x16

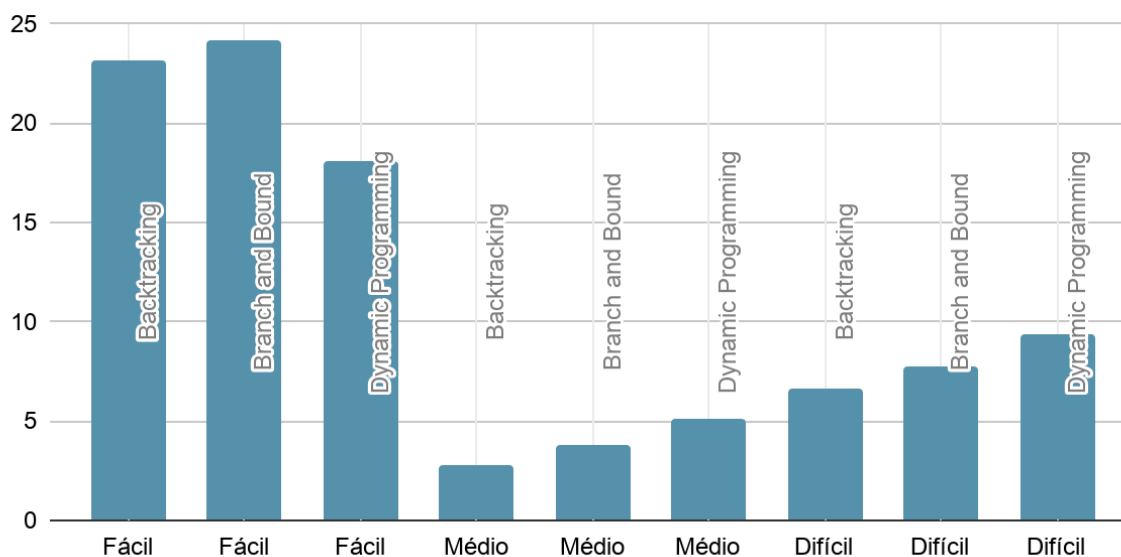


Gráfico 6 - Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 16x16.

Dificuldade	Algoritmo	Tempo médio (ms)	Desvio padrão (ms)	Nós visitados	Backtracks	Podas	Memória usada (MB)*
Fácil (prof. 147)	Backtracking	11,233	1,816	9714	9566	0	16,47
	Branch and Bound	51,782	7,643	527	695	316	22,93
	Dynamic Programming	42,168	10,746	9714	9566	0	48,54
Médio (prof. 73/72)*	Backtracking	104,738	32,969	100000†	99933	0	58,24
	Branch and Bound	9999,911	0,609	30816	51353	20606	75,21
	Dynamic Programming	434,337	60,154	100000†	99932	0	234,25
Difícil (prof. 72/50)*	Backtracking	114,830	20,132	100000†	99929	0	164,58
	Branch and Bound	9999,924	0,868	6866	11543	4725	224,77
	Dynamic Programming	393,439	94,156	100000†	99928	0	245,35

Tabela 6 - Resultados do Sudoku 25x25 (fácil, médio e difícil)

**A profundidade máxima do Branch and Bound difere das demais técnicas nas instâncias de médio/difícil porque a busca foi interrompida antes de esgotar o orçamento de nós ou tempo. Foi determinado um limite de nós configurado no experimento e quando atingido o limite a busca foi interrompida sem encontrar solução, e não corresponde ao tamanho real do espaço de busca (†).*

Comparação do tempo e desvio padrão em ms dos resultados obtidos do tabuleiro 25x25

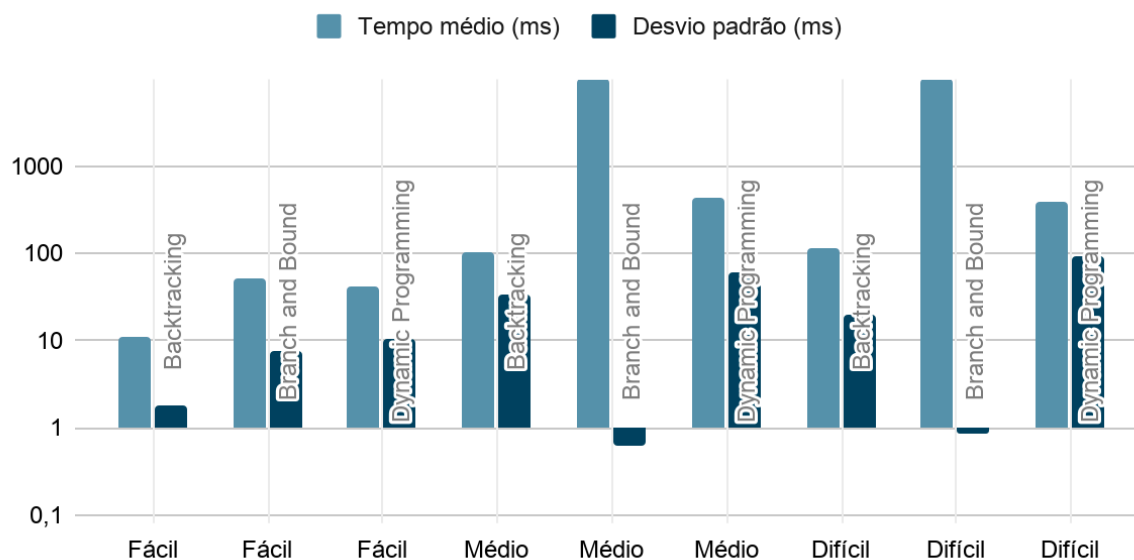


Gráfico 7 - Comparação do tempo e desvio padrão em ms dos resultados obtidos do tabuleiro 25x25

Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 25x25

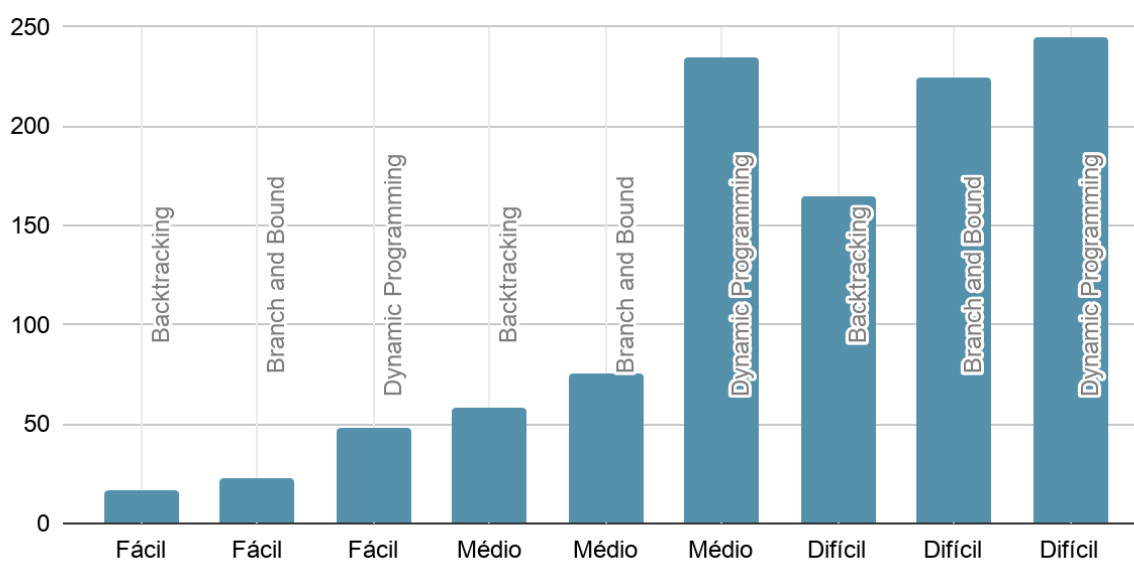


Gráfico 8 - Comparação do consumo de memória em MB dos resultados obtidos do tabuleiro 25x25

Na comparação de tabuleiro 4x4 os resultados obtidos nas três dificuldades, nenhum algoritmo registrou backtracks, ou seja, os tabuleiros foram resolvidos sem nenhuma tentativa incorreta. Como os tabuleiros 4x4 utilizados foram resolvidos sem necessidade de retrocesso, o espaço de busca efetivamente explorado foi pequeno, o número de nós visitados é praticamente igual entre os três algoritmos, e as pequenas diferenças de tempo (todas na casa dos centésimos de milissegundo) refletem apenas o overhead de implementação de cada técnica, não uma vantagem real de poda ou memoização.

Já no tabuleiro 9x9 a poda do Branch and Bound reduziu de forma consistente o número de nós visitados em relação ao Backtracking (32 contra 70 no fácil, 39 contra 86 no médio, 52 contra 85 no difícil), mas esse ganho não se traduziu em menor tempo: o Backtracking foi de 2 a 5 vezes mais rápido em todas as três dificuldades. A Programação Dinâmica visitou exatamente os mesmos nós que o Backtracking, sem nenhuma economia, e ainda assim foi mais lenta, evidenciando o custo de manter e consultar o conjunto de estados mortos sem contrapartida de poda nessas instâncias.

No tabuleiro de 16x16 é possível perceber que o padrão observado no 9x9 se intensifica: o Branch and Bound chega a ser quase 28 vezes mais lento que o Backtracking no nível difícil (9,353 ms contra 0,332 ms), mesmo tendo um número de nós visitados semelhante ou menor, o que indica que sua função de bound tem um custo por nó elevado. A Programação Dinâmica se mantém relativamente próxima do Backtracking em tempo, mas seu desvio padrão elevado no nível fácil (0,947 ms) sugere maior variabilidade de execução para execução.

No entanto, no tabuleiro 25x25 este é o ponto em que a poda do Branch and Bound se torna expressiva em número de nós (527 contra 9714 do Backtracking no fácil, uma redução de 94%), mas ainda assim seu tempo absoluto continua sendo o maior dos três, mostrando que o custo da função de bound por nó supera a economia de nós mesmo nessa escala. Nas instâncias médio e difícil, nenhum dos algoritmos convergem: Backtracking e Programação Dinâmica esgotam o limite de 100.000 nós, enquanto o Branch and Bound esgota o limite de tempo antes disso os três falham, mas por critérios diferentes.

5.3 Resultados da escalabilidade entre tamanhos de tabuleiro

Para isolar o efeito do crescimento do tabuleiro, a Tabela 7 consolida o tempo médio de execução dos três algoritmos no nível difícil de cada tamanho, permitindo observar a taxa de crescimento de cada técnica à medida que n aumenta de 4x4 para 25x25.

Algoritmo	4x4 (ms)	9x9 (ms)	16x16 (ms)	25x25 (ms)
Backtracking	0,017	0,076	0,332	114,830
Branch and Bound	0,020	0,367	9,353	9999,924*
Dynamic Programming	0,019	0,160	0,368	393,439

*Tabela 7 – Tempo médio (ms) no nível difícil, por algoritmo e tamanho de tabuleiro. *Branch and Bound em 25x25/difícil não concluiu a busca; o valor reflete o limite de tempo do experimento, não o tempo real de resolução.*

Comparação do tempo médio de execução do algoritmo em ms, entre todos os tabuleiros, no nível difícil

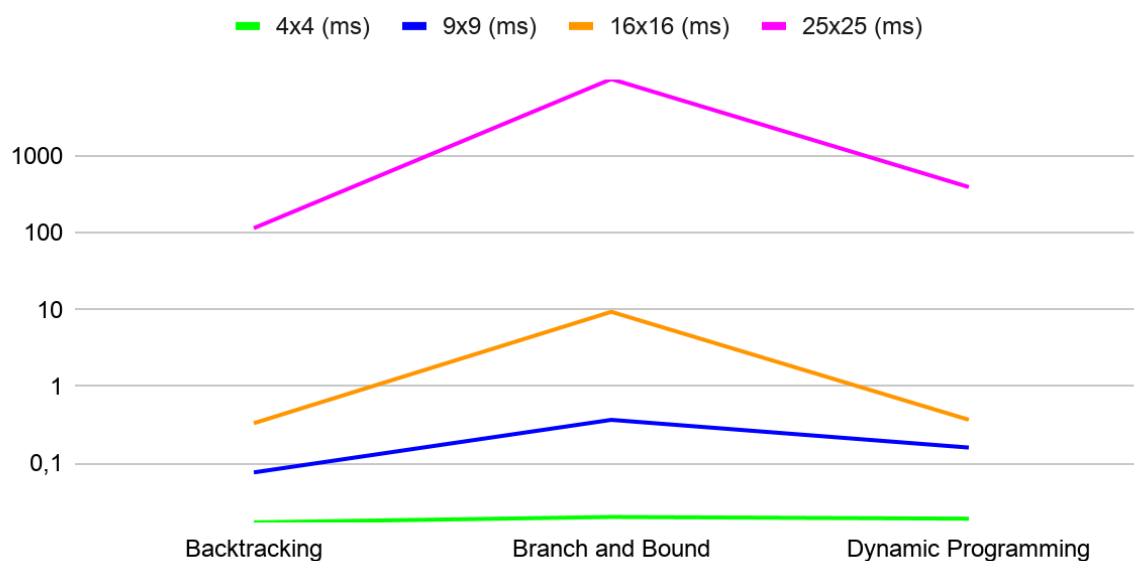


Gráfico 9 - Comparação do tempo médio de execução do algoritmo em ms, entre todos os tabuleiros, no nível difícil.

Fica evidente que os três algoritmos escalam de formas muito diferentes: o tempo do Backtracking cresce de forma relativamente “tranquila” até o 16x16, enquanto o do Branch and Bound dispara a partir desse mesmo ponto, chegando a ser o único limitado por tempo (e não por número de nós) no 25x25.

Abaixo, apresenta-se a tabela para mostrar o limite de escalabilidade dos resultados do tabuleiro de 25x25 nos níveis médio e difícil. Nas instâncias médio e difícil do tamanho 25x25, nenhum dos três algoritmos encontrou solução dentro dos limites configurados para o experimento (100.000 nós ou aproximadamente 10 segundos de execução). A Tabela 8 resume o critério de parada atingido por cada algoritmo nessas duas instâncias.

Algoritmo	Critério de parada atingido	Nós visitados (média)	Tempo (ms) médio	Taxa de sucesso
Backtracking	Limite de 100.000 nós	100.000	104,7 (médio) / 114,8 (difícil)	0/10
Branch and Bound	Limite de tempo (~10 s)	30.816 (médio) / 6.866 (difícil)	~9999,9	0/10
Dynamic Programming	Limite de 100.000 nós	100.000	434,3 (médio) / 393,4 (difícil)	0/10

Tabela 8 – Critério de parada atingido pelos três algoritmos em Sudoku 25x25, níveis médio e difícil, e sua taxa de sucesso.

A tabela acima mostra que a inviabilidade em 25x25 médio/difícil não tem uma única causa: Backtracking e Programação Dinâmica são interrompidos por excesso de nós, enquanto o Branch and Bound é interrompido por tempo, apesar de ter explorado uma fração muito menor do espaço de busca graças à sua poda.

5.4 Resultados de comparação do consumo de memória

A memória medida experimentalmente não deve ser confundida com a complexidade espacial assintótica. A primeira representa o consumo observado pela JVM durante a execução, enquanto a segunda descreve como o espaço necessário cresce teoricamente em função do tamanho da entrada.

Tamanho / Dificuldade	Backtracking (MB)	Branch and Bound (MB)	Dynamic Programming (MB)	Menor memória bruta reportada
4x4 – Fácil	10,96	11,53	12,11	Backtracking
4x4 – Médio	12,68	13,25	13,81	Backtracking
4x4 – Difícil	14,37	14,91	15,50	Backtracking
9x9 – Fácil	16,12	16,85	17,60	Backtracking
9x9 – Médio	18,47	19,11	19,95	Backtracking
9x9 – Difícil	20,71	21,43	22,27	Backtracking
16x16 – Fácil	23,20	24,14	18,12	Dynamic Programming†
16x16 – Médio	2,79†	3,80	5,16	Backtracking†
16x16 – Difícil	6,66	7,74	9,38	Backtracking
25x25 – Fácil	16,47	22,93	48,54	Backtracking
25x25 – Médio	58,24	75,21	234,25	Backtracking
25x25 – Difícil	164,58	224,77	245,35	Backtracking

Tabela 9 – Memória usada (MB) por algoritmo em cada combinação de tamanho e dificuldade, com destaque para o menor valor bruto reportado por instância.

Obs: A instância afetada por coleta de lixo da JVM durante a medição é marcada por †. Assim, nessas duas linhas, a comparação bruta não deve ser lida como diferença real de eficiência de memória.

Comparação do consumo de memória em MB, entre todos os tabuleiros e com todas as dificuldades de níveis.

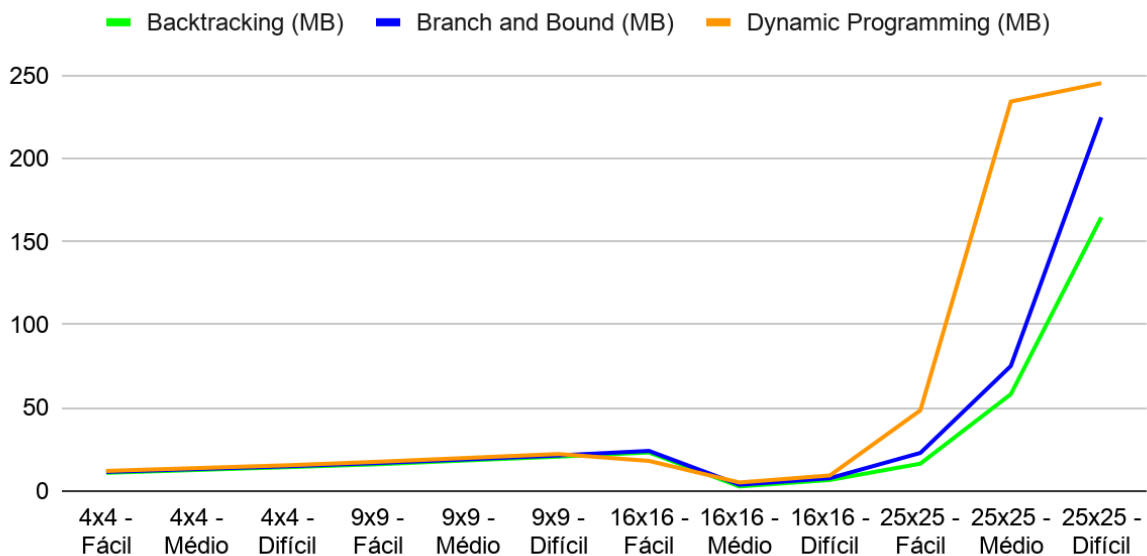


Gráfico 10 - Comparação do consumo de memória em MB, entre todos os tabuleiros e com todas as dificuldades de níveis.

Os valores de memória obtidos devem ser interpretados com cautela, pois a métrica utilizada está sujeita ao gerenciamento automático de memória da JVM. Ainda assim, os resultados brutos indicam uma tendência de menor consumo para o Backtracking e maior consumo para a Programação Dinâmica, especialmente nas instâncias maiores.

6. ANÁLISE E DISCUSSÃO

Esta seção apresenta e discute os resultados obtidos nos experimentos com os algoritmos Backtracking, Branch and Bound e Programação Dinâmica, considerando diferentes tamanhos e níveis de dificuldade dos tabuleiros de Sudoku. A análise utiliza as métricas coletadas durante as execuções para comparar o desempenho das abordagens e identificar como o aumento da complexidade dos tabuleiros influencia seus resultados.

6.1 Comparação do Tempo de Execução

Em relação ao tempo de execução, o Backtracking apresentou o melhor desempenho na maior parte das combinações de tamanho e dificuldade avaliadas. Essa vantagem foi observada tanto em tabuleiros pequenos quanto nas instâncias maiores que foram concluídas, incluindo o Sudoku 25×25 no nível fácil.

A Programação Dinâmica apresentou desempenho intermediário na maioria das instâncias, com tempos próximos aos do Backtracking nos tabuleiros menores. Entretanto, à medida que o tamanho do tabuleiro aumentou, o custo associado à memoização tornou-se mais perceptível. No Sudoku 25×25 fácil, por exemplo, a Programação Dinâmica apresentou tempo médio de 42,168 ms, enquanto o Backtracking apresentou 11,233 ms.

O Branch and Bound apresentou uma característica diferente. Embora tenha reduzido significativamente a quantidade de nós visitados em várias instâncias, esse benefício não se refletiu diretamente no tempo de execução. No Sudoku 16×16 difícil, por exemplo, o Branch and Bound visitou 193 nós, contra 197 do Backtracking, mas apresentou tempo médio de 9,353 ms, enquanto o Backtracking apresentou 0,332 ms.

Esse comportamento evidencia que a quantidade de nós explorados não é suficiente, isoladamente, para determinar o desempenho de um algoritmo. O custo das operações realizadas em cada nó também exerce influência significativa sobre o tempo total de execução. No caso do Branch and Bound, a execução da função de bound adiciona um custo considerável à busca.

Assim, considerando o tempo de execução como principal critério de desempenho, o Backtracking apresentou a melhor eficiência entre as técnicas avaliadas nas condições experimentais deste trabalho.

6.2 Comparação do Consumo de Memória

Em relação ao consumo de memória, os resultados devem ser interpretados com cautela, pois a métrica utilizada representa o consumo observado pela JVM durante a execução e pode ser influenciada pelo gerenciamento automático de memória e pela coleta de lixo.

Ainda assim, os valores apresentados na Tabela 9 permitem observar algumas tendências. O Backtracking apresentou, em geral, valores inferiores aos demais algoritmos nas instâncias menores, enquanto a Programação Dinâmica apresentou valores mais elevados em diversas instâncias maiores. Esse comportamento é compatível com a estrutura das implementações: o Backtracking utiliza principalmente a própria representação do tabuleiro e a pilha de recursão, enquanto a Programação Dinâmica mantém adicionalmente um conjunto de estados mortos utilizados na memoização.

O Branch and Bound também apresenta estruturas auxiliares relacionadas ao armazenamento e à avaliação dos candidatos e ao mecanismo de bound, o que acrescenta custo de memória em relação ao Backtracking simples.

Entretanto, as diferenças observadas não devem ser interpretadas como medidas exatas da complexidade espacial de cada algoritmo. Os valores absolutos podem sofrer influência do estado do heap da JVM no momento da medição. Dessa

forma, a análise de memória deve ser considerada principalmente em conjunto com a complexidade espacial teórica apresentada na Fundamentação Teórica, e não apenas pela comparação direta dos valores em megabytes.

6.3 Corretude e Convergência das Soluções

Em relação à corretude, os três algoritmos apresentaram o mesmo comportamento nas instâncias que foram concluídas. Todos produziram soluções válidas para os tabuleiros testados, confirmando experimentalmente as propriedades de corretude discutidas na Fundamentação Teórica.

No contexto do Sudoku, não existe uma noção de solução ótima associada a um valor de custo a ser minimizado ou maximizado. O objetivo é encontrar uma atribuição completa que satisfaça todas as restrições do problema. Dessa forma, a comparação entre os algoritmos deve considerar principalmente sua capacidade de encontrar uma solução válida e o custo necessário para obtê-la.

Nas instâncias que convergiram, os três algoritmos apresentaram taxa de sucesso de 10/10 execuções. Isso ocorreu para os tamanhos de 4×4 , 9×9 e 16×16 em todos os níveis de dificuldade, além do 25×25 no nível fácil.

Nas instâncias 25×25 de dificuldade média e difícil, nenhum dos algoritmos conseguiu concluir a busca dentro dos limites experimentais estabelecidos. Nessas situações, a taxa de sucesso foi de 0/10 para os três algoritmos.

Portanto, os resultados indicam que as diferenças entre as técnicas não estão relacionadas à qualidade da solução obtida, mas principalmente ao custo computacional necessário para encontrá-la e à capacidade de concluir a busca dentro dos recursos disponíveis.

6.4 Limites de Viabilidade dos Algoritmos

Os resultados indicam que os três algoritmos conseguiram concluir as instâncias de 4×4 , 9×9 e 16×16 nos três níveis de dificuldade avaliados. Também houve convergência para a instância 25×25 de dificuldade fácil.

O limite de viabilidade conjunta ocorreu nas instâncias 25×25 de dificuldade média e difícil. Nesses casos, nenhum dos três algoritmos encontrou uma solução dentro dos limites experimentais estabelecidos, correspondendo a uma taxa de sucesso de 0/10.

Apesar disso, os algoritmos não apresentaram o mesmo comportamento à medida que se aproximaram desse limite. O Branch and Bound apresentou degradação de desempenho mais acentuada em termos de tempo. No Sudoku 16×16 difícil, por exemplo, seu tempo médio foi de 9,353 ms, enquanto o Backtracking apresentou 0,332 ms e a Programação Dinâmica 0,368 ms.

Dessa forma, é possível distinguir dois aspectos diferentes da viabilidade. O primeiro está relacionado à capacidade de concluir a resolução dentro dos limites estabelecidos, enquanto o segundo está relacionado ao custo necessário para realizar essa resolução. Sob esse segundo critério, o Branch and Bound apresentou perda de desempenho antes dos demais algoritmos.

Assim, o tamanho 25×25 , especialmente nos níveis médio e difícil, representa o principal limite de escalabilidade observado nos experimentos realizados.

6.5 Impacto do Crescimento do Tamanho do Tabuleiro

O aumento do tamanho do tabuleiro produziu impacto significativo no

desempenho dos três algoritmos, principalmente nas instâncias de maior dificuldade.

Considerando o nível difícil, o tempo médio do Backtracking passou de 0,017 ms no Sudoku 4×4 para 0,076 ms no 9×9, 0,332 ms no 16×16 e 114,830 ms no 25×25. Esse comportamento evidencia um aumento expressivo do custo computacional no maior tamanho avaliado, associado principalmente ao crescimento do espaço de busca.

O Branch and Bound apresentou crescimento ainda mais acentuado do tempo de execução. Seu tempo passou de 0,020 ms no 4×4 para 0,367 ms no 9×9, 9,353 ms no 16×16 e aproximadamente 10 segundos no 25×25 difícil, quando a execução foi interrompida pelo limite de tempo estabelecido.

A Programação Dinâmica apresentou crescimento intermediário no tempo de execução, passando de 0,019 ms no 4×4 para 0,160 ms no 9×9, 0,368 ms no 16×16 e 393,439 ms no 25×25 difícil. Além do aumento do tempo, sua estratégia de memoização também implica maior consumo de memória à medida que mais estados são armazenados.

Os resultados demonstram, portanto, que o aumento de N não afeta apenas a quantidade de estados que podem ser explorados, mas também o custo das operações realizadas em cada estado. Esse efeito explica parte das diferenças observadas entre as técnicas e reforça a importância da análise de escalabilidade realizada neste trabalho.

6.6 Relação entre Complexidade Teórica e Desempenho Prático

Os resultados evidenciam uma diferença importante entre a análise assintótica e o comportamento observado experimentalmente. Embora os três algoritmos apresentem crescimento exponencial no pior caso, a notação assintótica não representa todos os custos constantes e operações adicionais realizadas

durante a execução.

O Branch and Bound ilustra claramente essa diferença. Em várias instâncias, o algoritmo visitou uma quantidade significativamente menor de nós que o Backtracking. No Sudoku 25×25 fácil, por exemplo, foram registrados 527 nós para o Branch and Bound e 9.714 para o Backtracking, representando uma redução expressiva na quantidade de estados explorados. Entretanto, o tempo médio foi de 51,782 ms para o Branch and Bound e 11,233 ms para o Backtracking.

Esse resultado demonstra que reduzir a quantidade de nós explorados não garante, isoladamente, menor tempo de execução. O custo necessário para calcular o bound em cada etapa pode ser suficientemente elevado para superar o benefício obtido pela poda.

De maneira semelhante, a Programação Dinâmica adiciona o custo de serialização dos estados e de manutenção do conjunto de estados mortos. Dessa forma, sua análise prática também depende da relação entre a quantidade de estados evitados e o custo necessário para armazená-los e consultá-los.

Assim, os experimentos reforçam que a análise assintótica é fundamental para compreender o comportamento de crescimento dos algoritmos, mas não é suficiente para prever, isoladamente, qual implementação apresentará menor tempo de execução em uma determinada instância.

6.7 Eficiência dos Mecanismos de Poda e Memoização

Os mecanismos de poda e memoização apresentaram efeitos distintos sobre a exploração do espaço de busca.

O Branch and Bound foi capaz de reduzir a quantidade de nós visitados em diversas instâncias. No Sudoku 9×9, por exemplo, foram registrados 32, 39 e 52 nós para as dificuldades fácil, média e difícil, respectivamente, enquanto o Backtracking visitou 70, 86 e 85 nós. No Sudoku 25×25 fácil, a diferença foi ainda maior, com 527

nós visitados pelo Branch and Bound contra 9.714 pelo Backtracking.

Apesar dessa redução, o custo adicional da função de bound impediu que a menor quantidade de nós se convertesse em menor tempo de execução. Assim, a poda mostrou-se eficiente na redução do espaço explorado, mas não necessariamente no tempo total de resolução nas condições avaliadas.

A Programação Dinâmica, por sua vez, não apresentou redução na quantidade de nós visitados nas instâncias analisadas em relação ao Backtracking. Isso indica que, nessas instâncias, a ocorrência de estados repetidos que poderiam ser eliminados pela memoização foi limitada ou inexistente. Como consequência, o custo adicional de serialização e armazenamento dos estados não foi compensado por uma redução equivalente da busca.

Portanto, os resultados mostram que a eficiência de uma estratégia de poda ou memoização depende não apenas de sua capacidade de eliminar estados, mas também do custo necessário para identificar e descartar esses estados.

6.8 Custo Computacional e Benefícios das Estratégias Algorítmicas

Os resultados indicam que, nas instâncias avaliadas, os custos adicionais introduzidos pelo Branch and Bound e pela Programação Dinâmica não foram compensados pelos benefícios obtidos em termos de redução do espaço de busca.

No Branch and Bound, o mecanismo de bound conseguiu reduzir significativamente o número de nós visitados, mas introduziu um custo adicional de processamento em cada etapa da busca. Como consequência, o algoritmo apresentou tempos superiores aos do Backtracking na maioria das instâncias.

Na Programação Dinâmica, a memoização também adicionou custos relacionados à serialização dos estados e ao armazenamento no conjunto de

estados mortos. Como os resultados não apresentaram redução significativa no número de nós visitados, esse custo adicional não proporcionou vantagem de tempo em relação ao Backtracking.

Esses resultados não significam que as técnicas de poda e memoização sejam inadequadas para problemas de busca. Eles indicam apenas que, nas instâncias e implementações avaliadas neste trabalho, os custos adicionais dessas estratégias foram superiores aos benefícios obtidos.

A utilização de estratégias mais avançadas de propagação de restrições e seleção de variáveis poderia produzir resultados diferentes em instâncias maiores, mas essa possibilidade não foi investigada experimentalmente neste projeto.

6.9 Limitações da Coleta e Interpretação dos Dados

Algumas limitações da coleta de dados devem ser consideradas na interpretação dos resultados. A principal delas está relacionada à métrica de memória, que foi obtida a partir do consumo observado pela JVM durante as execuções. Como o gerenciamento de memória é realizado automaticamente pela JVM, os valores podem sofrer influência da coleta de lixo e do estado do heap no momento da medição. Dessa forma, os valores absolutos de memória não devem ser interpretados como uma medida exata da complexidade espacial dos algoritmos.

Outra limitação está relacionada às execuções interrompidas pelos limites experimentais. Nas instâncias 25×25 de dificuldade média e difícil, alguns algoritmos atingiram o limite de 100.000 nós, enquanto o Branch and Bound atingiu o limite de tempo estabelecido. Nesses casos, os tempos e quantidades de nós representam o comportamento observado até o momento da interrupção e não o custo necessário para concluir a resolução do Sudoku.

Também foi identificada uma inconsistência no registro da quantidade de execuções não concluídas para o Branch and Bound nas instâncias 25×25 média e difícil. Embora a taxa de sucesso registrada seja 0/10, o contador de execuções não concluídas não apresenta a mesma quantidade. Esse dado deve ser revisado na

implementação da coleta antes de ser utilizado como métrica definitiva.

Essas limitações não invalidam a comparação geral realizada, mas indicam que determinados resultados devem ser interpretados com cautela, principalmente aqueles relacionados ao consumo absoluto de memória e às execuções que foram interrompidas pelos limites experimentais.

7. CONCLUSÃO

Este trabalho apresentou a implementação e a análise comparativa de três técnicas exatas de projeto de algoritmos Backtracking, Branch and Bound e Programação Dinâmica aplicadas à resolução do Sudoku em quatro tamanhos de tabuleiro (4x4, 9x9, 16x16 e 25x25) e três níveis de dificuldade, totalizando doze combinações distintas de instância avaliadas de forma cruzada entre os algoritmos. A generalização do sistema para múltiplas dimensões, e não apenas o tabuleiro 9x9 tradicional, permitiu examinar como o comportamento relativo dos algoritmos muda continuamente à medida que o tamanho do tabuleiro cresce, o que se mostrou o fator mais informativo dos experimentos.

Os resultados mostram que a vantagem teórica das técnicas com poda e memoização não se manifestou em tempo de execução para a grande maioria das instâncias testadas: o Backtracking, o algoritmo mais simples e com menor custo por nó, foi o mais rápido em tempo absoluto em quase todos os tamanhos e dificuldades, apesar de visitar sistematicamente mais nós que o Branch and Bound. A exceção relevante não foi uma inversão desse padrão, mas sim seu colapso conjunto: em 25x25 médio e difícil, nenhum dos três algoritmos foi capaz de concluir a busca dentro dos limites de tempo e nós estabelecidos, revelando que, para tabuleiros grandes com poucas pistas, nenhuma das três estratégias implementadas é suficiente isoladamente, e que técnicas adicionais de propagação de restrições provavelmente seriam necessárias para tornar essas instâncias tratáveis.

O achado mais importante em termos de escalabilidade foi o descolamento nas taxas de crescimento do tempo de execução entre algoritmos: enquanto o Backtracking apresentou crescimento moderado até 16x16, seguido de um aumento muito acentuado no 25x25; e o Branch and Bound apresentou crescimento muito mais acentuado a partir do 16x16, tornando-se o único algoritmo limitado por tempo (em vez de por número de nós) em 25x25. Isso demonstra, de forma concreta, que o custo por nó de um mecanismo de poda não é uma constante desprezível: ele pode escalar com o tamanho do tabuleiro a uma taxa maior que a própria redução no

número de nós que ele proporciona, invertendo na prática a vantagem que a análise assintótica sugere.

Em consumo de memória, o Backtracking também se mostrou a alternativa mais econômica na quase totalidade das instâncias, condizente com sua ausência de estruturas auxiliares, enquanto a Programação Dinâmica foi consistentemente a mais custosa, em linha com o espaço adicional exigido pelo conjunto de estados mortos que mantém. Combinando tempo, memória e taxa de sucesso, o Backtracking foi a opção mais eficiente em quase todo o espaço de instâncias testado, ao passo que o Branch and Bound e a Programação Dinâmica só justificariam seu custo adicional em cenários com espaço de busca substancialmente maior do que os suportados atualmente pela aplicação cenário sugerido, mas não confirmado, pelo colapso conjunto observado em 25x25 médio e difícil.

A principal lição deste trabalho é, portanto, que a escolha do algoritmo mais adequado para resolver Sudoku depende criticamente do tamanho da instância, e que decisões de projeto validadas apenas em tabuleiros pequenos podem não se sustentar à medida que o problema escala.

Desafio Extra (Bônus)

Além dos requisitos centrais do projeto, foram implementadas três funcionalidades adicionais previstas na lista de desafios extras da disciplina:

A Interface gráfica para execução dos algoritmos foi desenvolvida uma aplicação gráfica em Java Swing (classe SudokuGUI), independente do menu de terminal, que permite selecionar o tamanho do tabuleiro, o algoritmo e o nível de dificuldade, executar a resolução e visualizar o tabuleiro preenchido junto das métricas coletadas.

Visualização gráfica da execução dos algoritmos: Para o Backtracking e o Branch and Bound em tabuleiros de até 9×9, a interface exibe a busca em tempo real por meio do padrão observador (PassoObservador): a cada tentativa, a célula avaliada é destacada visualmente na grade e o valor testado é registrado no painel "Passo a passo", com a execução do algoritmo rodando em uma thread separada (SwingWorker) para não bloquear a interface durante a animação.

Framework de benchmark: A classe PerformanceComparisonTest automatiza a execução cruzada dos três algoritmos em todas as combinações de tamanho de tabuleiro (4×4, 9×9, 16×16 e 25×25) e nível de dificuldade (fácil, médio e difícil), repetindo cada configuração 10 vezes para reduzir a influência de variações momentâneas. Os resultados de cada execução tempo, memória, nós visitados, chamadas recursivas, backtracks, podas e profundidade máxima são exportados automaticamente para um arquivo CSV pela classe ResultsExporter, servindo de base para as tabelas e análises apresentadas neste relatório.

REFERÊNCIAS

DA CRUZ, Lucas Ribeiro et al. RECONHECIMENTO DE DÍGITOS EM TABULEIROS DE SUDOKU COM VISÃO COMPUTACIONAL E RESOLUÇÃO POR FORÇA BRUTA. REVISTA FOCO , v. 6, pág. e8944-e8944, 2025. Disponível em: <<https://doi.org/10.54751/revistafoco.v18n6-206>>. Acesso em: 12/08/2026.

EFFENDI, Rustam; GUNAWAN, Indra; EFENDI, Yasin. Projeto de Software para Conclusão do Jogo Sudoku com Algoritmo Branch and Bound. In: 1ª Conferência Internacional Multidisciplinar sobre Educação, Tecnologia e Engenharia (IMCETE 2019) . Atlantis Press, 2020. p. 126-130. Disponível em: <<https://www.atlantis-press.com/proceedings/imcete-19/125935522>>. Acesso em: 10/08/2026.

TAKANO, Kevin; DE FREITAS, Rosiane; DE SÁ, Vinícius. O jogo de lógica sudoku: modelagem teórica, np-completude e estratégias algorítmicas exatas e heurísticas. In: Concurso de Trabalhos de Iniciação Científica da SBC (CTIC-SBC). SBC, 2015. p. 71-80.

Disponível em: <<https://sol.sbc.org.br/index.php/ctic/article/view/10020>>. Acesso em: 17/08/2026.

Link do repositório: https://github.com/ellerimx/sudokuSolver_projetoTAAL.git